PYTHON ACTIVITY BASED LEARNING

Case Study Report

Name : MEHAVARSHINI

Roll No : 26001600

Branch : AIML

Section : 1

Selected Case Studies

Case Study Number	Title	Concepts
3	Hospital Billing and Patient Classification	Variables, arithmetic, conditions
4	Employee Payroll and Deduction Analysis	Arithmetic, conditions
7	Retail Billing with Multiple Discount Rules	Lists, loops, conditions
11	Examination Result and Grace-Mark Validation	Lists, conditions
13	ATM Transaction Sequence Simulation	Loops, lists, conditions
14	Warehouse Inventory Reconciliation	Lists, arithmetic
17	Student Performance Trend Analysis	Lists, comparisons
28	Repeated Transaction Detection	Lists, loops
42	Student Marks Matrix Analysis	Nested lists, loops
49	Tuple-Based Product Catalogue Audit	Tuples, loops

CASE 01: Hospital Billing

1. Problem Statement

Hospitals bill patients for several services, and calculating the final amount by hand can lead to mistakes. This project develops a Python program that accepts the consultation, diagnostic, and treatment charges of a patient, validates them, and calculates the total bill. A concession is applied when the total reaches a fixed limit, and the final bill is then classified as a Low, Medium, or High bill.

2. Objectives

1. To develop a Python program for hospital bill calculation.
2. To accept consultation, diagnostic, and treatment charges as input.
3. To reject invalid (negative) charges.
4. To calculate the total bill.
5. To apply a concession when the total bill is Rs. 5000 or more.
6. To calculate the final bill after the concession.
7. To classify the final bill as Low, Medium, or High and display the result.

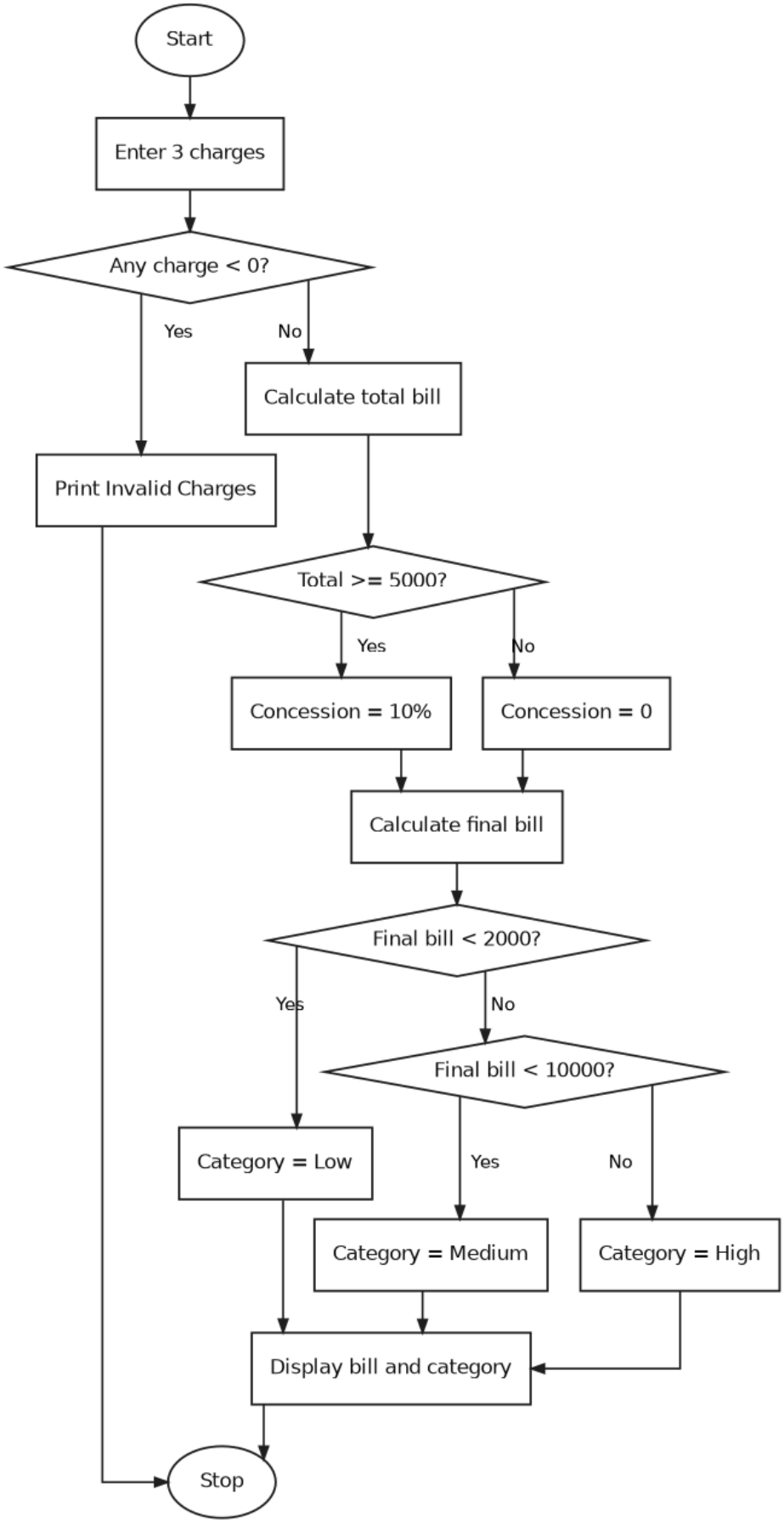
3. Assumptions

1. The three charges are entered as numbers (decimal values are allowed).
2. No charge may be negative; a zero charge is accepted.
3. Total bill = consultation + diagnostic + treatment.
4. A concession of 10% of the total bill is given if the total bill is Rs. 5000 or more; otherwise there is no concession.
5. Final bill = total bill - concession.
6. Final bill below Rs. 2000 is a **Low Bill**.
7. Final bill from Rs. 2000 up to (but not including) Rs. 10000 is a **Medium Bill**.
8. Final bill of Rs. 10000 or more is a **High Bill**.
9. These are sample rules for learning; they are not official hospital rates.

4. Algorithm

1. Start.
2. Enter the consultation, diagnostic, and treatment charges.
3. If any charge is less than 0, display "Invalid charges" and stop.
4. Calculate total = consultation + diagnostic + treatment.
5. If total >= 5000, calculate concession = total x 10 / 100; otherwise set concession = 0.
6. Calculate final bill = total - concession.
7. If final bill < 2000, set category to "Low Bill".
8. Otherwise, if final bill < 10000, set category to "Medium Bill".
9. Otherwise, set category to "High Bill".
10. Display the total bill, concession, final bill, and category.
11. Stop.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

INPUT consultation, diagnostic, treatment

IF consultation < 0 OR diagnostic < 0 OR treatment < 0
    DISPLAY "Invalid charges"
ELSE
    total = consultation + diagnostic + treatment

    IF total >= 5000
        concession = total * 10 / 100
    ELSE
        concession = 0
    END IF

    final_bill = total - concession

    IF final_bill < 2000
        category = "Low Bill"
    ELSE IF final_bill < 10000
        category = "Medium Bill"
    ELSE
        category = "High Bill"
    END IF

    DISPLAY total, concession, final_bill, category
END IF

STOP
```

7. Python Source Code / Program

```
consultation = float(input("Enter consultation charge: "))
diagnostic = float(input("Enter diagnostic charge: "))
treatment = float(input("Enter treatment charge: "))

if consultation < 0 or diagnostic < 0 or treatment < 0:
    print("Invalid charges")
else:
    total = consultation + diagnostic + treatment

    if total >= 5000:
        concession = total * 10 / 100
    else:
        concession = 0

    final_bill = total - concession

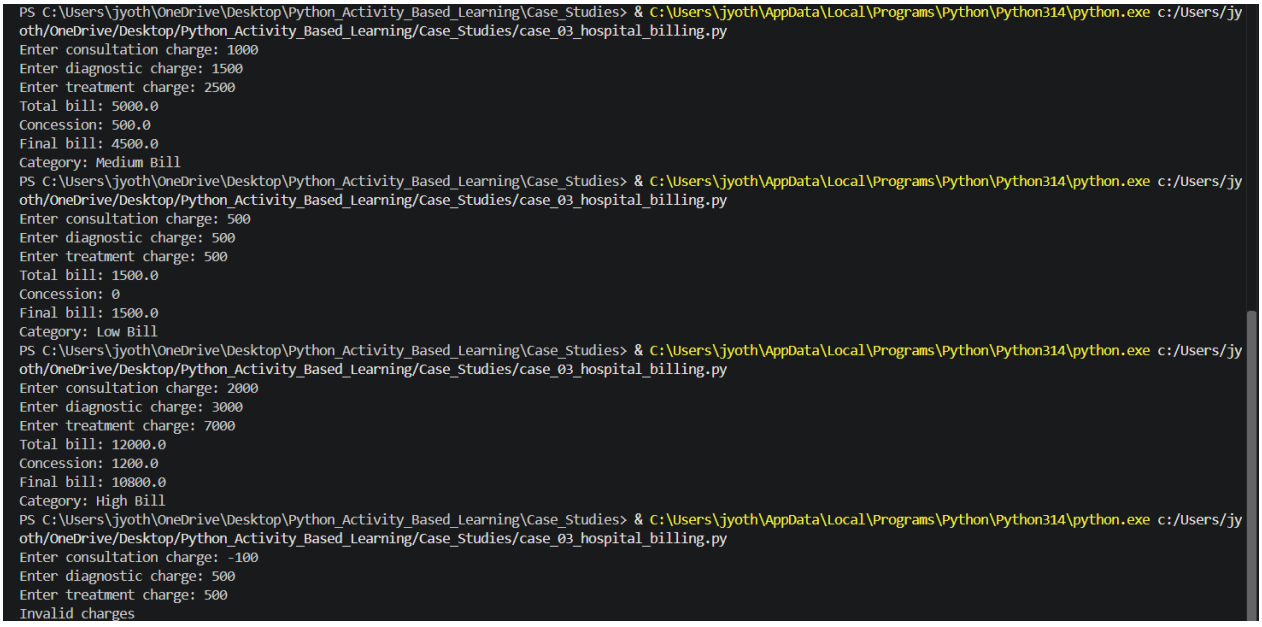
    if final_bill < 2000:
        category = "Low Bill"
    elif final_bill < 10000:
        category = "Medium Bill"
    else:
        category = "High Bill"

    print("Total bill:", total)
    print("Concession:", concession)
    print("Final bill:", final_bill)
    print("Category:", category)
```

8. Test Cases

Test	Consultation	Diagnostic	Treatment	Expected result
1	1000	1500	2500	Total 5000; concession 500; final 4500; Medium Bill
2	500	500	500	Total 1500; no concession; final 1500; Low Bill
3	2000	3000	7000	Total 12000; concession 1200; final 10800; High Bill
4	-100	500	500	Invalid charges
5	2000	2000	999	Total 4999; no concession (below 5000); final 4999; Medium Bill
6	700	700	600	Total 2000; no concession; final 2000; Medium Bill (boundary)

9. Output Screenshot



10. Conclusion

This program calculates a patient's hospital bill from the consultation, diagnostic, and treatment charges. It rejects invalid negative charges, applies a 10% concession on bills of Rs. 5000 or more, and classifies the final bill as Low, Medium, or High. The test runs confirm that the calculation and the classification work correctly.

CASE 02: Manufacturing Cost and Profitability Analysis

1. Problem Statement

A manufacturing unit needs to know whether producing a product is profitable. Calculating costs and profit by hand for different production quantities is slow and error-prone. This project develops a Python program that calculates the material, labour, and machine costs for a product, finds the total manufacturing cost and total sales revenue, calculates the profit, and checks whether the profit target has been achieved for several production quantities.

2. Objectives

1. To develop a Python program for manufacturing cost and profitability analysis.
2. To calculate the total material, labour, and machine cost for a given quantity.
3. To calculate the total manufacturing cost.
4. To calculate the total sales revenue from the selling price.
5. To calculate the profit as revenue minus total cost.
6. To check whether the profit target is achieved.
7. To test the program with five different production quantities.

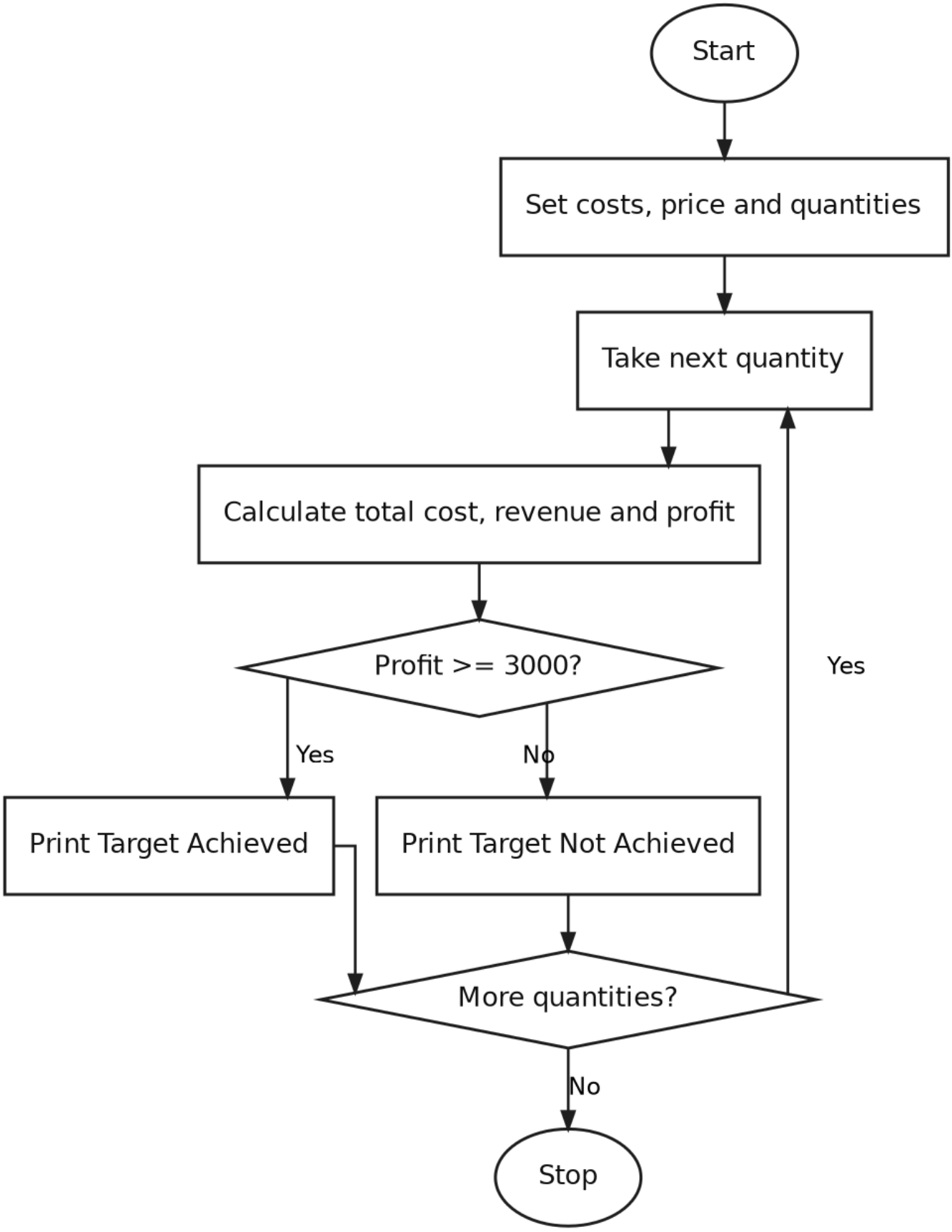
3. Assumptions

1. The product is School Bags.
2. Material cost is Rs. 120, labour cost is Rs. 30, and machine cost is Rs. 10 per unit.
3. The selling price is Rs. 200 per unit.
4. The profit target is Rs. 3000.
5. Total manufacturing cost = quantity x (material + labour + machine cost per unit).
6. Total sales revenue = quantity x selling price; all units produced are sold.
7. Profit = total sales revenue - total manufacturing cost.
8. The profit target is achieved when profit is greater than or equal to Rs. 3000.
9. The test quantities are 100, 50, 75, 25, and 150 units.
10. These are sample values for learning; they are not real company figures.

4. Algorithm

1. Start.
2. Set the product name, unit costs, selling price, profit target, and the list of test quantities.
3. Take the next quantity from the list.
4. Calculate total material cost, total labour cost, and total machine cost for that quantity.
5. Calculate total cost = material + labour + machine cost.
6. Calculate total revenue = quantity x selling price.
7. Calculate profit = total revenue - total cost.
8. Display the product name, quantity, total cost, total revenue, and profit.
9. If profit >= profit target, display "Profit Target: Achieved"; otherwise display "Profit Target: Not Achieved".
10. Display a separator line and repeat from step 3 until all quantities are processed.
11. Stop.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

SET product_name = "School Bags"
SET material_cost = 120
SET labour_cost = 30
SET machine_cost = 10
SET selling_price = 200
SET profit_target = 3000
SET test_quantities = [100, 50, 75, 25, 150]

FOR each quantity in test_quantities:
    total_material_cost = quantity * material_cost
    total_labour_cost = quantity * labour_cost
    total_machine_cost = quantity * machine_cost

    total_cost = total_material_cost + total_labour_cost
                 + total_machine_cost
    total_revenue = quantity * selling_price
    profit = total_revenue - total_cost

    DISPLAY product_name, quantity, total_cost,
            total_revenue, profit

    IF profit >= profit_target
        DISPLAY "Profit Target: Achieved"
    ELSE
        DISPLAY "Profit Target: Not Achieved"
    END IF

    DISPLAY separator line
END FOR

STOP
```

7. Python Source Code / Program

```
# Manufacturing Cost and Profitability Analysis

product_name = "School Bags"

material_cost = 120
labour_cost = 30
machine_cost = 10
selling_price = 200
profit_target = 3000

# Five test cases
test_quantities = [100, 50, 75, 25, 150]

for quantity in test_quantities:
    total_material_cost = quantity * material_cost
    total_labour_cost = quantity * labour_cost
    total_machine_cost = quantity * machine_cost

    total_cost = (
        total_material_cost
        + total_labour_cost
        + total_machine_cost
    )

    total_revenue = quantity * selling_price
    profit = total_revenue - total_cost

    print("\nProduct Name:", product_name)
    print("Quantity:", quantity)
    print("Total Manufacturing Cost:", total_cost)
    print("Total Sales Revenue:", total_revenue)
    print("Profit:", profit)

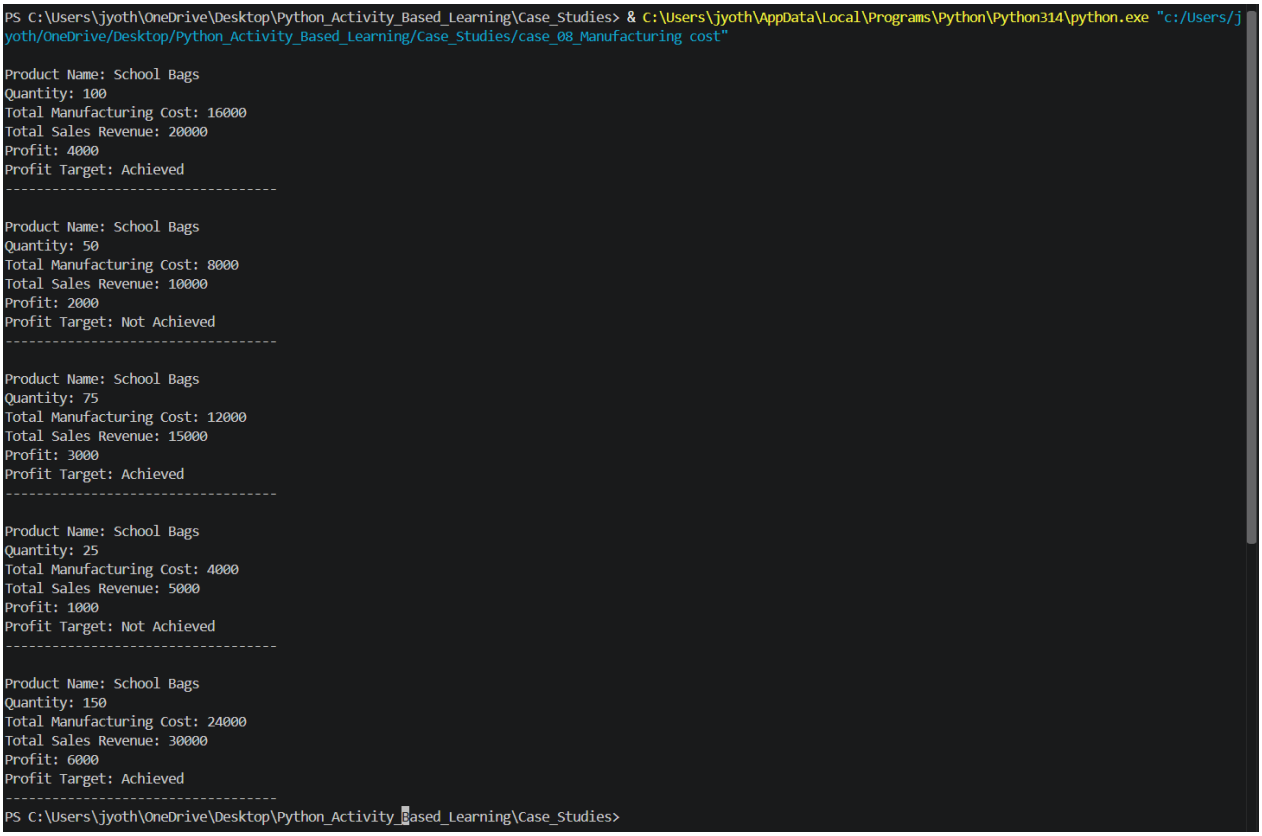
    if profit >= profit_target:
        print("Profit Target: Achieved")
    else:
        print("Profit Target: Not Achieved")

    print("-" * 35)
```

8. Test Cases

Test	Quantity	Total cost	Revenue	Profit	Expected result
1	100	16000	20000	4000	Profit Target: Achieved
2	50	8000	10000	2000	Profit Target: Not Achieved
3	75	12000	15000	3000	Profit Target: Achieved (profit equals target)
4	25	4000	5000	1000	Profit Target: Not Achieved
5	150	24000	30000	6000	Profit Target: Achieved

9. Output Screenshot



10. Conclusion

This program calculates the material, labour, and machine costs for different production quantities, finds the total manufacturing cost and sales revenue, and computes the profit. By comparing the profit with the target of Rs. 3000, it shows that the target is achieved for 75 units and above, and not achieved for 50 and 25 units. This helps in planning production quantities.

CASE 03: ATM Transaction Sequence Simulation

1. Problem Statement

An ATM must process a sequence of customer transactions correctly, updating the account balance only when a transaction is valid. This project develops a Python program that simulates a series of deposit and withdrawal transactions. Each transaction is checked for a valid amount, the withdrawal limit, and sufficient balance. The program updates the balance after every successful transaction, rejects invalid ones with a reason, and displays a final summary of deposits, withdrawals, and successful and rejected transactions.

2. Objectives

1. To develop a Python program that simulates a sequence of ATM transactions.
2. To process deposit and withdrawal transactions one after another.
3. To reject transactions with an invalid (zero or negative) amount.
4. To reject withdrawals that exceed the withdrawal limit or the available balance.
5. To update the account balance after each successful transaction.
6. To display the status and current balance after every transaction.
7. To display the total deposits, total withdrawals, number of successful and rejected transactions, and the final balance.

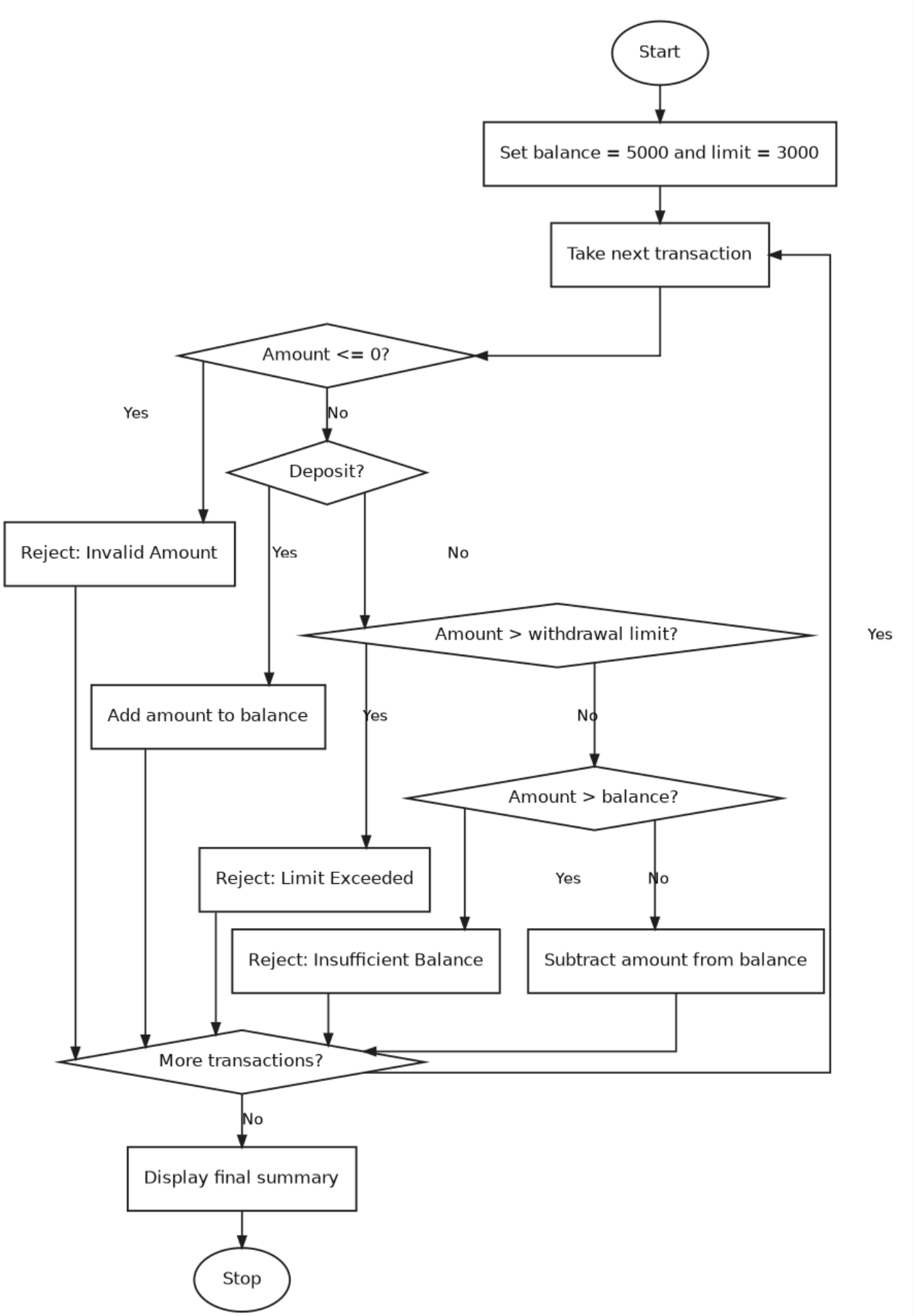
3. Assumptions

1. The starting balance is Rs. 5000.
2. The maximum amount for a single withdrawal is Rs. 3000.
3. The transactions are given as a list of (transaction type, amount) pairs and are processed in the given order.
4. The five test transactions are: Deposit 1000, Withdraw 2000, Withdraw 4000, Deposit 500, and Withdraw -200.
5. An amount less than or equal to zero is invalid and the transaction is rejected.
6. A deposit with a valid amount is always accepted.
7. A withdrawal is accepted only if the amount is within the withdrawal limit and does not exceed the balance.
8. A rejected transaction does not change the balance.
9. These are sample values for learning; they are not real bank rules.

4. Algorithm

1. Start.
2. Set balance = 5000, withdrawal limit = 3000, and the list of transactions; set all counters to zero.
3. Display the ATM transaction summary heading and the starting balance.
4. Take the next transaction from the list and display its type and amount.
5. If the amount is less than or equal to 0, reject the transaction as an invalid amount.
6. Otherwise, if the transaction is a Deposit, add the amount to the balance and total deposits, and count it as successful.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

SET balance = 5000
SET withdrawal_limit = 3000
SET transactions = [("Deposit", 1000), ("Withdraw", 2000),
                    ("Withdraw", 4000), ("Deposit", 500),
                    ("Withdraw", -200)]
SET total_deposits = 0, total_withdrawals = 0
SET successful = 0, rejected = 0

DISPLAY heading and starting balance

FOR each (transaction, amount) in transactions:
    DISPLAY transaction and amount

    IF amount <= 0
        DISPLAY "Rejected - Invalid amount"
        rejected = rejected + 1
    ELSE IF transaction = "Deposit"
        balance = balance + amount
        total_deposits = total_deposits + amount
        successful = successful + 1
        DISPLAY "Successful"
    ELSE IF transaction = "Withdraw"
        IF amount > withdrawal_limit
            DISPLAY "Rejected - Exceeds withdrawal limit"
            rejected = rejected + 1
        ELSE IF amount > balance
            DISPLAY "Rejected - Insufficient balance"
            rejected = rejected + 1
        ELSE
            balance = balance - amount
            total_withdrawals = total_withdrawals + amount
            successful = successful + 1
            DISPLAY "Successful"
        END IF
    END IF

    DISPLAY current balance
END FOR

DISPLAY total_deposits, total_withdrawals,
        successful, rejected, balance

STOP
```

7. Python Source Code / Program

```
# ATM Transaction Sequence Simulation
balance = 5000
withdrawal_limit = 3000

transactions = [
    ("Deposit", 1000),
    ("Withdraw", 2000),
    ("Withdraw", 4000),
    ("Deposit", 500),
    ("Withdraw", -200)
]

total_deposits = 0
total_withdrawals = 0
successful = 0
rejected = 0

print("ATM TRANSACTION SUMMARY")
print("Starting Balance: Rs.", balance)
print("-" * 35)

for transaction, amount in transactions:
    print("\nTransaction:", transaction)
    print("Amount: Rs.", amount)

    if amount <= 0:
        print("Status: Rejected - Invalid amount")
        rejected += 1

    elif transaction == "Deposit":
        balance += amount
        total_deposits += amount
        successful += 1
        print("Status: Successful")

    elif transaction == "Withdraw":
        if amount > withdrawal_limit:
            print("Status: Rejected - Exceeds withdrawal limit")
            rejected += 1
        elif amount > balance:
            print("Status: Rejected - Insufficient balance")
            rejected += 1
        else:
            balance -= amount
            total_withdrawals += amount
            successful += 1
            print("Status: Successful")

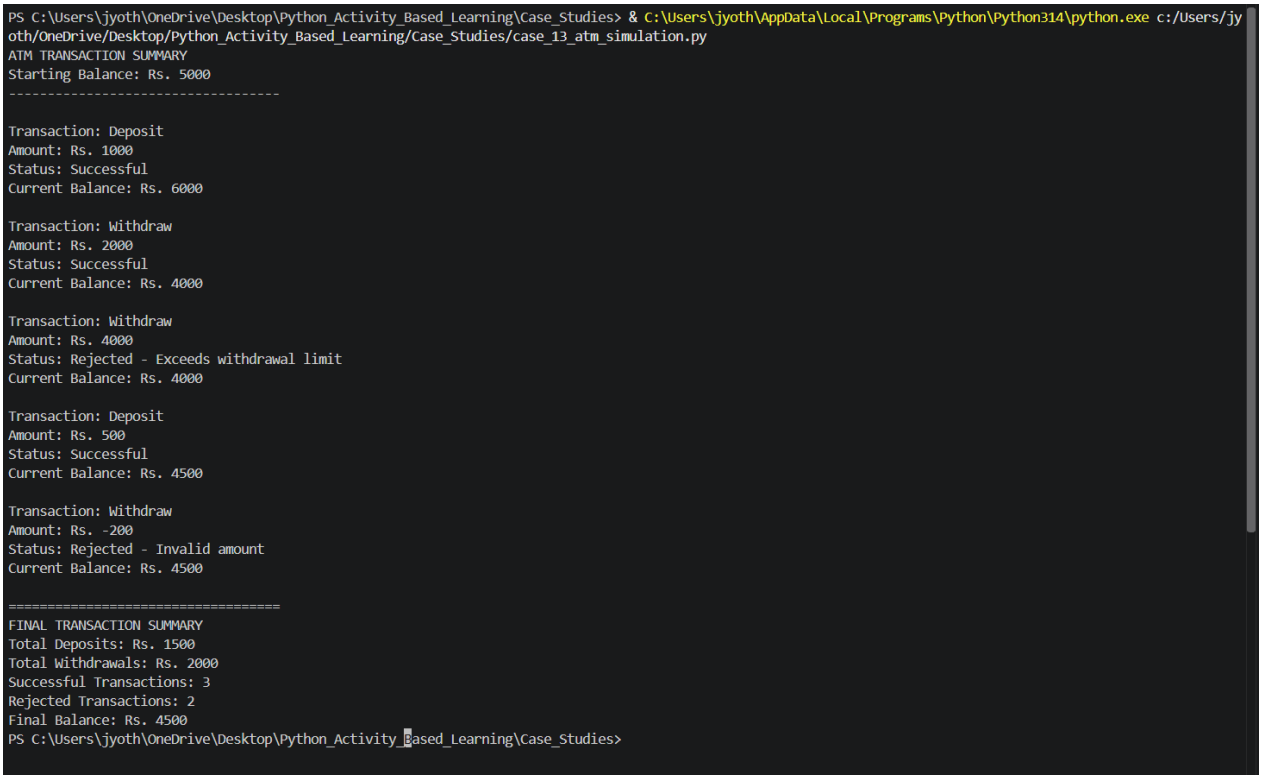
    print("Current Balance: Rs.", balance)

print("\n" + "=" * 35)
print("FINAL TRANSACTION SUMMARY")
print("Total Deposits: Rs.", total_deposits)
print("Total Withdrawals: Rs.", total_withdrawals)
print("Successful Transactions:", successful)
print("Rejected Transactions:", rejected)
print("Final Balance: Rs.", balance)
```

8. Test Cases

Test	Transaction	Amount (Rs.)	Expected result	Balance (Rs.)
1	Deposit	1000	Successful	6000
2	Withdraw	2000	Successful	4000
3	Withdraw	4000	Rejected - Exceeds withdrawal limit	4000
4	Deposit	500	Successful	4500
5	Withdraw	-200	Rejected - Invalid amount	4500
6	Withdraw	0	Rejected - Invalid amount	No change
7	Withdraw	3000 (balance 2500)	Rejected - Insufficient balance	No change
8	Final summary after tests 1-5	-	Deposits 1500; withdrawals 2000; successful 3; rejected 2	4500

9. Output Screenshot



10. Conclusion

This program simulates a sequence of ATM transactions. It validates each amount, enforces the withdrawal limit and the available balance, updates the balance only for successful transactions, and reports the reason for every rejection. The final summary of 3 successful and 2 rejected transactions with a final balance of Rs. 4500 matches the expected results.

CASE 04: Student Marks Matrix Analysis

1. Problem Statement

A teacher needs to analyse the marks of several students in more than one subject. Calculating totals, averages, results, and the best performer by hand is slow and can lead to mistakes. This project develops a Python program that stores the marks of students in a matrix (a list of lists), calculates each student's total and average marks, decides whether the student has passed or failed, and finds the student with the highest average marks.

2. Objectives

1. To develop a Python program that stores student marks in a matrix.
2. To calculate the total marks of each student across three subjects.
3. To calculate the average marks of each student.
4. To decide the result (Pass or Fail) of each student from the average marks.
5. To find the student with the highest average marks.
6. To display the marks, total, average, and result of every student, followed by the highest scoring student.

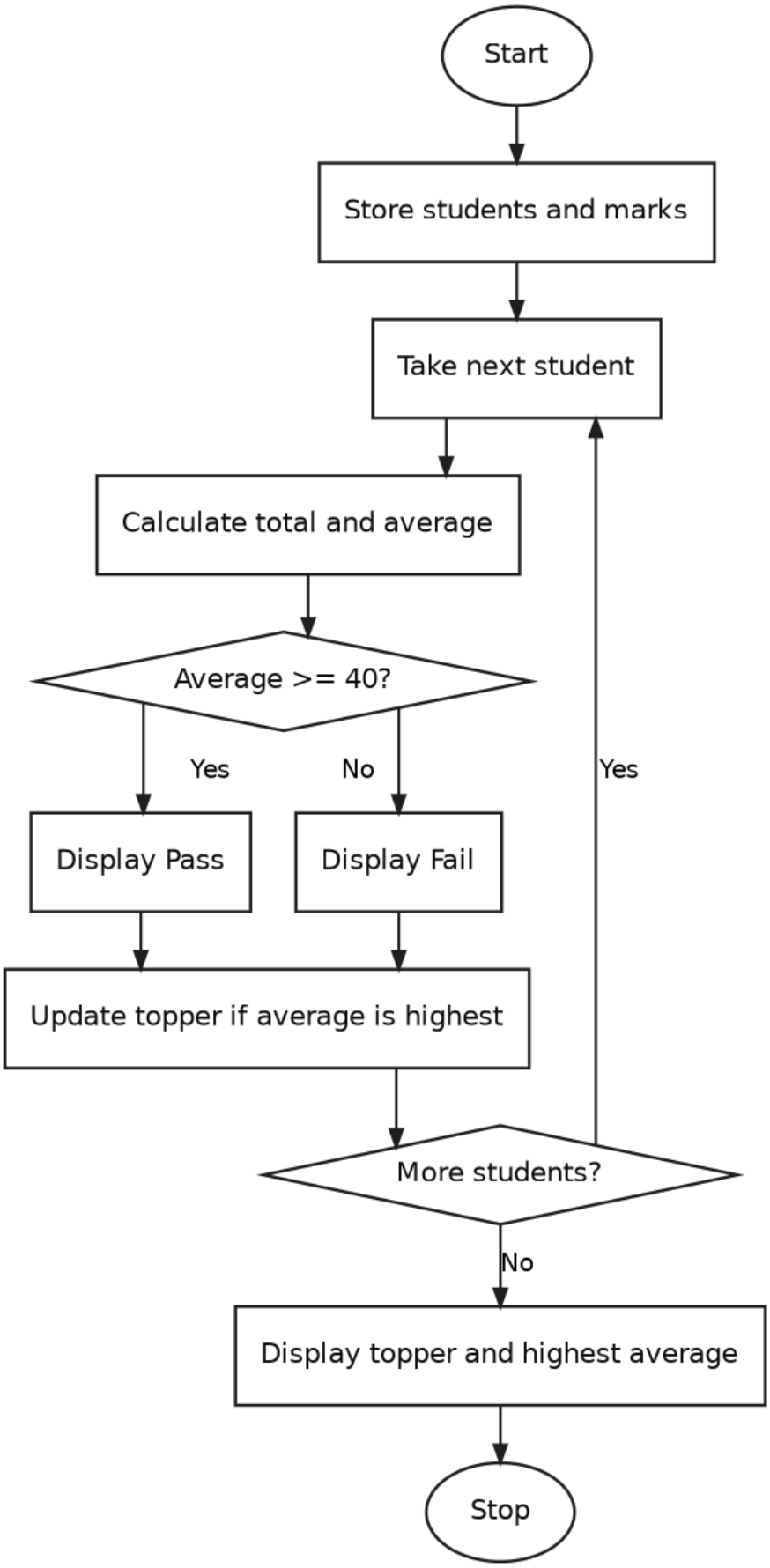
3. Assumptions

1. There are five students: Ravi, Priya, Raju, Sita, and Kiran.
2. Each student has marks in three subjects: Python, Maths, and English.
3. The marks matrix has one row per student and columns Python, Maths, English.
4. Marks are valid whole numbers out of 100.
5. Total marks = Python + Maths + English marks.
6. Average = total / number of subjects (shown to 2 decimal places).
7. A student passes if the average marks are 40 or more; otherwise the student fails.
8. The topper has the highest average; on a tie, the first student is kept.
9. These are sample marks for learning; they are not real student data.

4. Algorithm

1. Start.
2. Store the student names and the marks matrix; set highest_average = -1 and topper = empty.
3. Take the next student from the list.
4. Calculate the total marks and the average marks of the student.
5. Display the student's name, the three subject marks, the total, and the average.
6. If the average is 40 or more, display "Result: Pass"; otherwise display "Result: Fail".
7. If the average is greater than highest_average, update highest_average and set topper to this student.
8. Repeat from step 4 until all students are processed.
9. Display the highest scoring student and the highest average.
10. Stop.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

SET students = ["Ravi", "Priya", "Raju", "Sita", "Kiran"]
SET marks = [[80, 75, 85],
             [90, 88, 92],
             [60, 65, 70],
             [95, 90, 98],
             [45, 50, 55]]
SET highest_average = -1
SET topper = ""

DISPLAY heading

FOR each student i:
    total = sum of marks[i]
    average = total / number of subjects

    DISPLAY name, Python marks, Maths marks,
            English marks, total, average

    IF average >= 40
        DISPLAY "Result: Pass"
    ELSE
        DISPLAY "Result: Fail"
    END IF

    IF average > highest_average
        highest_average = average
        topper = students[i]
    END IF
END FOR

DISPLAY topper
DISPLAY highest_average

STOP
```

7. Python Source Code / Program

```
# Student Marks Matrix Analysis
students = ["Ravi", "Priya", "Raju", "Sita", "Kiran"]
# Marks in 3 subjects: Python, Maths, English
marks = [
    [80, 75, 85],
    [90, 88, 92],
    [60, 65, 70],
    [95, 90, 98],
    [45, 50, 55]
]
print("STUDENT MARKS MATRIX ANALYSIS")
print("-" * 50)

highest_average = -1
topper = ""
for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(marks[i])

    print("\nStudent Name:", students[i])
    print("Python Marks:", marks[i][0])
    print("Maths Marks:", marks[i][1])
    print("English Marks:", marks[i][2])
    print("Total Marks:", total)
    print("Average Marks:", round(average, 2))

    if average >= 40:
        print("Result: Pass")
    else:
        print("Result: Fail")

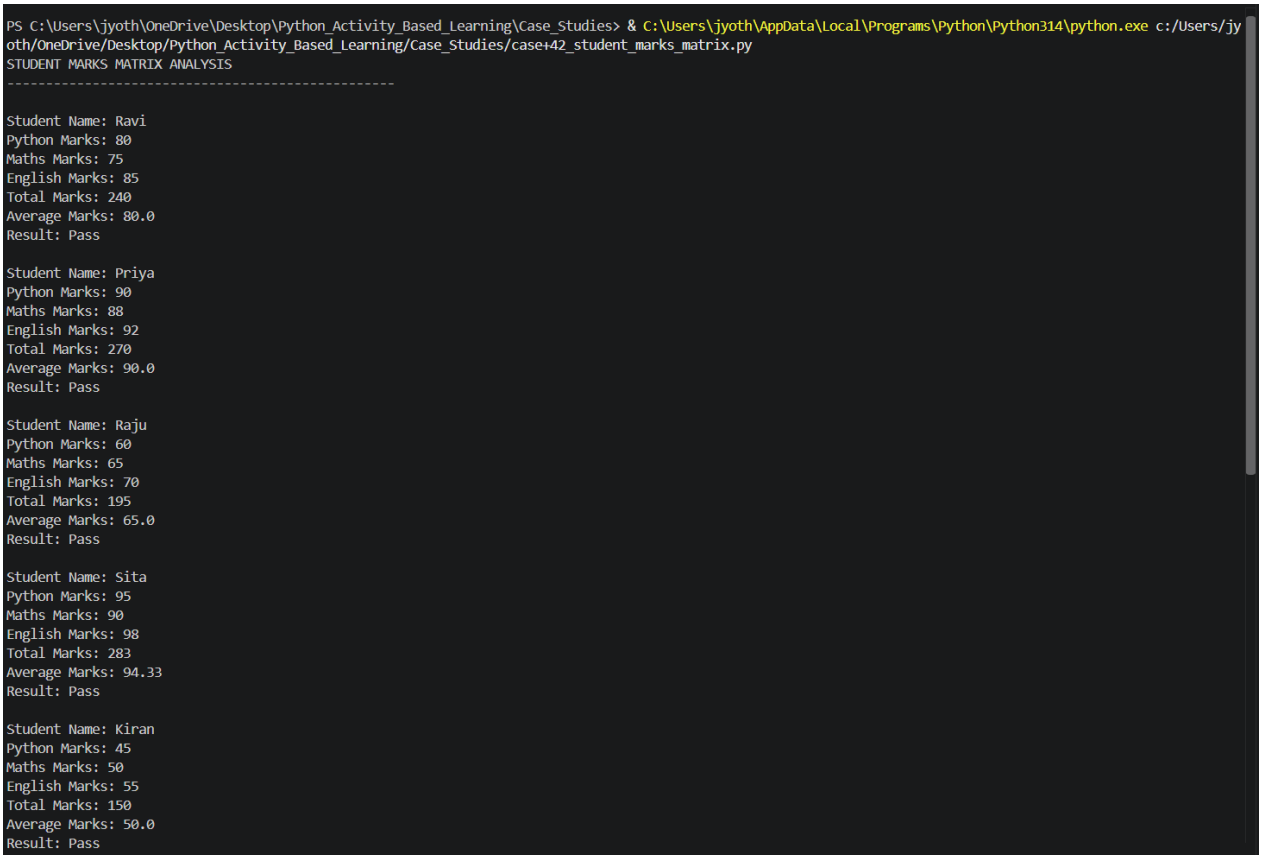
    if average > highest_average:
        highest_average = average
        topper = students[i]

print("\n" + "-" * 50)
print("Highest Scoring Student:", topper)
print("Highest Average:", round(highest_average, 2))
```

8. Test Cases

Test	Student	Python	Maths	English	Total	Average	Result
1	Ravi	80	75	85	240	80.0	Pass
2	Priya	90	88	92	270	90.0	Pass
3	Raju	60	65	70	195	65.0	Pass
4	Sita	95	90	98	283	94.33	Pass
5	Kiran	45	50	55	150	50.0	Pass
6	Boundary	40	40	40	120	40.0	Pass (average equals 40)
7	Low marks	40	40	39	119	39.67	Fail
8	Highest scoring student (tests 1-5)	-	-	-	-	94.33	Sita

9. Output Screenshot



10. Conclusion

This program stores student marks in a matrix and uses a loop to calculate the total, average, and result of every student. It also tracks the highest average to identify the topper. For the sample data, all five students pass and Sita is the highest scoring student with an average of 94.33.

CASE 5: Tuple-Based Transaction Audit

1. Problem Statement

Banks and payment systems process a large number of transactions every day, and each record must be checked before it is accepted. Invalid amounts, unsupported transaction types, and repeated transaction IDs can lead to accounting errors and financial loss. This project develops a Python program that stores each transaction as a tuple (transaction ID, amount, transaction type), audits every record, labels it as valid, invalid, or duplicate, and displays a summary of the audit.

2. Objectives

1. To develop a Python program that audits a list of transactions stored as tuples.
2. To unpack each tuple into transaction ID, amount, and transaction type.
3. To detect duplicate transaction IDs using a list of IDs already seen.
4. To identify invalid amounts (zero or negative values).
5. To identify invalid transaction types other than "Deposit" and "Withdraw".
6. To display the status of every transaction and count valid, invalid, and duplicate records.

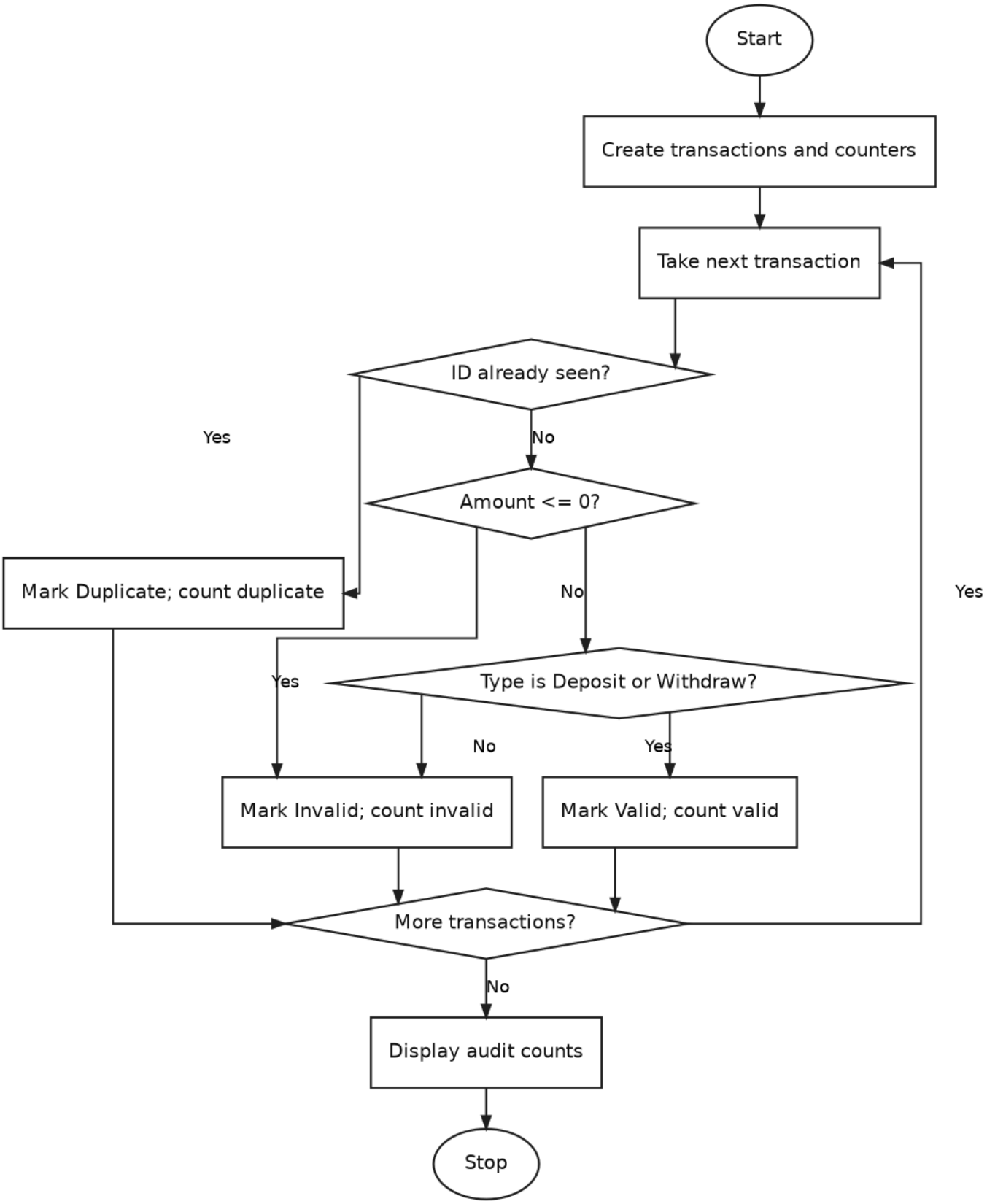
3. Assumptions

1. Each transaction is a tuple in the form (transaction_id, amount, transaction_type).
2. The allowed transaction types are "Deposit" and "Withdraw" (case-sensitive).
3. A valid amount must be greater than zero.
4. The first occurrence of a transaction ID is processed; later records with the same ID are duplicates.
5. The duplicate check is performed first, before the amount and type checks.
6. IDs of valid and invalid transactions are stored in `seen_ids`; duplicate records are not stored again.
7. Each transaction is counted in exactly one category: valid, invalid, or duplicate.
8. Amounts are in rupees (Rs.), and the transaction data is predefined in the program.

4. Algorithm

1. Start the program.
2. Create the list of transaction tuples and an empty list `seen_ids`.
3. Set valid_count, invalid_count, and duplicate_count to zero.
4. Take the next transaction and unpack it into ID, amount, and type; display the details.
5. If the ID is already in seen_ids, display "Duplicate Transaction ID" and increase duplicate_count.
6. Otherwise, if the amount is less than or equal to zero, display "Invalid Amount", increase invalid_count, and store the ID.
7. Otherwise, if the type is not "Deposit" or "Withdraw", display "Invalid Transaction Type", increase invalid_count, and store the ID.
8. Otherwise, display "Valid Transaction", increase valid_count, and store the ID.
9. Repeat steps 4 to 8 for all transactions.
10. Display the valid, invalid, duplicate, and total transaction counts.
11. Stop the program.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

CREATE list of transactions (ID, amount, type)
CREATE empty list seen_ids
SET valid_count = 0, invalid_count = 0, duplicate_count = 0

FOR each transaction in transactions:
    UNPACK transaction into ID, amount, type
    DISPLAY ID, amount, type

    IF ID is in seen_ids:
        DISPLAY "Duplicate Transaction ID"
        duplicate_count = duplicate_count + 1
    ELSE IF amount <= 0:
        DISPLAY "Invalid Amount"
        invalid_count = invalid_count + 1
        ADD ID to seen_ids
    ELSE IF type is not "Deposit" and not "Withdraw":
        DISPLAY "Invalid Transaction Type"
        invalid_count = invalid_count + 1
        ADD ID to seen_ids
    ELSE:
        DISPLAY "Valid Transaction"
        valid_count = valid_count + 1
        ADD ID to seen_ids

DISPLAY valid_count, invalid_count, duplicate_count
DISPLAY total number of transactions

STOP
```

7. Python Source Code / Program

```
# Tuple-Based Transaction Audit
transactions = [
    (101, 500, "Deposit"),
    (102, 1000, "Withdraw"),
    (103, -200, "Deposit"),
    (102, 300, "Deposit"),
    (104, 700, "Withdraw")
]
seen_ids = []
valid_count = 0
invalid_count = 0
duplicate_count = 0
print("TUPLE-BASED TRANSACTION AUDIT")
print("-" * 40)
for transaction in transactions:
    transaction_id, amount, transaction_type = transaction
    print("\nTransaction ID:", transaction_id)
    print("Amount: Rs.", amount)
    print("Type:", transaction_type)
    if transaction_id in seen_ids:
        print("Status: Duplicate Transaction ID")
        duplicate_count += 1
    elif amount <= 0:
        print("Status: Invalid Amount")
        invalid_count += 1
        seen_ids.append(transaction_id)
    elif transaction_type not in ("Deposit", "Withdraw"):
        print("Status: Invalid Transaction Type")
        invalid_count += 1
        seen_ids.append(transaction_id)
    else:
        print("Status: Valid Transaction")
        valid_count += 1
        seen_ids.append(transaction_id)
print("\n" + "-" * 40)
print("Valid Transactions:", valid_count)
print("Invalid Transactions:", invalid_count)
print("Duplicate Transactions:", duplicate_count)
print("Total Transactions:", len(transactions))
```

8. Test Cases

Test case	Transaction (ID, amount, type)	Input condition	Expected result
TC01	(101, 500, "Deposit")	New ID, positive amount, valid type	Valid Transaction
TC02	(102, 1000, "Withdraw")	New ID, positive amount, valid type	Valid Transaction
TC03	(103, -200, "Deposit")	New ID, negative amount	Invalid Amount
TC04	(102, 300, "Deposit")	ID 102 already processed	Duplicate Transaction ID
TC05	(104, 700, "Withdraw")	New ID, positive amount, valid type	Valid Transaction

Overall expected result: Valid = 3, Invalid = 1, Duplicate = 1, Total = 5.

9. Output Screenshot

```
PS C:\Users\jyoth\OneDrive\Desktop\Python_Activity_Based_Learning\Case_Studies> & C:\Users\jyoth\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jyoth/OneDrive/Desktop/Python_Activity_Based_Learning/Case_Studies/case_55_tuple_based_transacation_audit.py
TUPLE-BASED TRANSACTION AUDIT
----------------------------------------

Transaction ID: 101
Amount: Rs. 500
Type: Deposit
Status: Valid Transaction

Transaction ID: 102
Amount: Rs. 1000
Type: Withdraw
Status: Valid Transaction

Transaction ID: 103
Amount: Rs. -200
Type: Deposit
Status: Invalid Amount

Transaction ID: 102
Amount: Rs. 300
Type: Deposit
Status: Duplicate Transaction ID

Transaction ID: 104
Amount: Rs. 700
Type: Withdraw
Status: Valid Transaction

----------------------------------------
Valid Transactions: 3
Invalid Transactions: 1
Duplicate Transactions: 1
Total Transactions: 5
PS C:\Users\jyoth\OneDrive\Desktop\Python_Activity_Based_Learning\Case_Studies>
```

10. Conclusion

This program audits a list of transactions stored as tuples. It unpacks each record, detects duplicate IDs, rejects non-positive amounts and unsupported transaction types, and reports the number of valid, invalid, and duplicate transactions. The output matches the expected results of all five test cases, showing how tuples, lists, and conditional checks can be combined to make transaction auditing consistent and easy to verify.

CASE 6: Student Academic Risk Classification

1. Problem Statement

Colleges need to identify students who may be at risk of poor academic performance so that support can be given early. Checking marks and attendance for every student manually is slow and may be inconsistent. This project develops a Python program that stores each student's record as a tuple (name, marks, attendance), applies predefined rules to marks and attendance, and classifies every student as High Risk, Medium Risk, or Low Risk.

2. Objectives

1. To develop a Python program that classifies students by academic risk level.
2. To store student records as tuples of name, marks, and attendance percentage.
3. To unpack each tuple and display the student's details.
4. To classify a student as High Risk when marks are below 40 or attendance is below 50%.
5. To classify a student as Medium Risk when marks are below 60 or attendance is below 75%.
6. To classify all remaining students as Low Risk and display the result for each student.

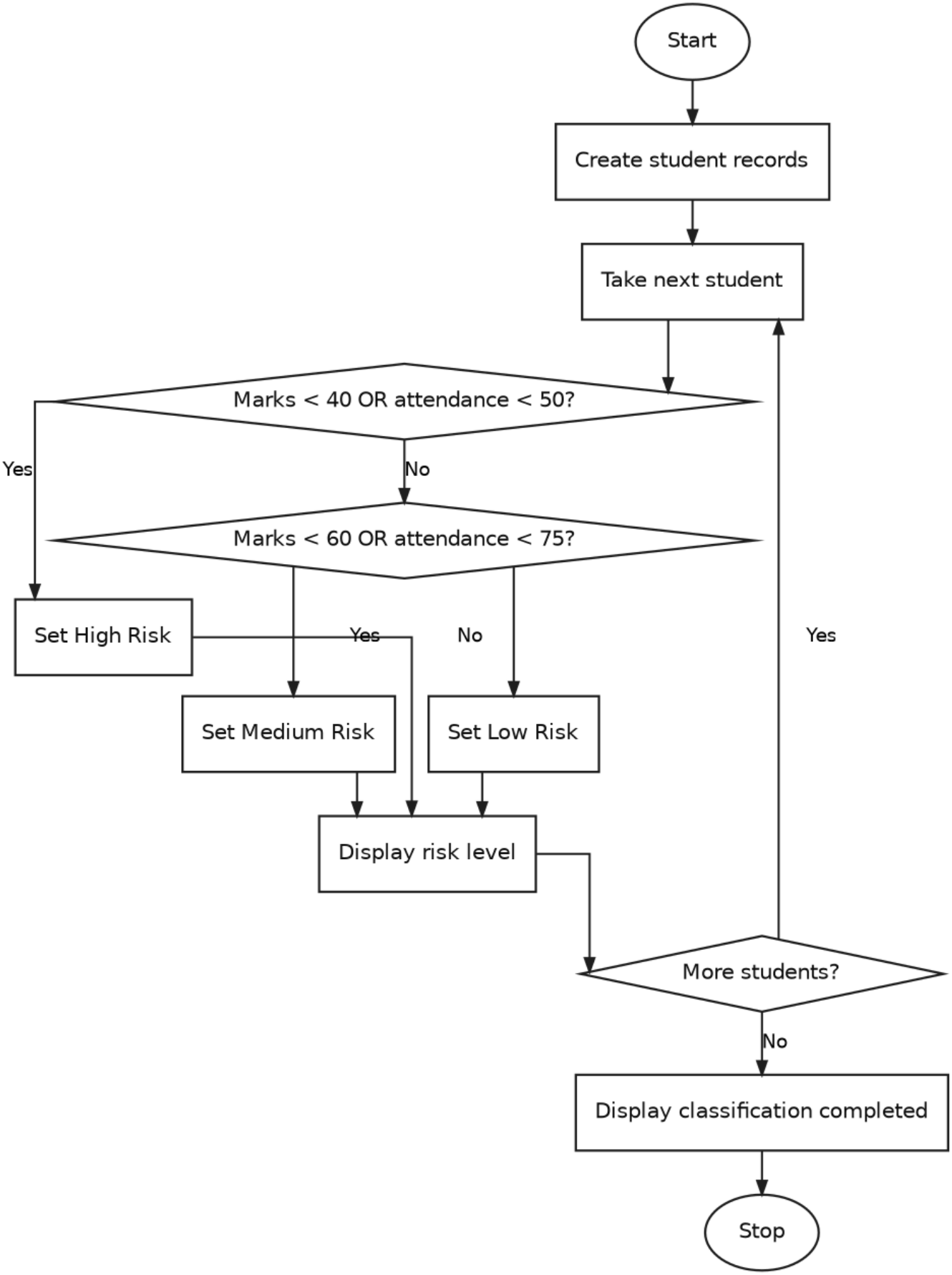
3. Assumptions

1. Each student record is a tuple in the form (name, marks, attendance).
2. Marks are out of 100 and attendance is a percentage from 0 to 100.
3. The student data is predefined in the program and is entered correctly.
4. High Risk: marks below 40 or attendance below 50%.
5. Medium Risk: not High Risk, but marks below 60 or attendance below 75%.
6. Low Risk: marks of 60 or more and attendance of 75% or more.
7. The conditions are checked in order (High, Medium, Low), so each student receives exactly one risk level.
8. These threshold values are assumed for educational purposes; actual institutional rules may differ.

4. Algorithm

1. Start the program.
2. Create the list of student tuples (name, marks, attendance).
3. Take the next student and unpack the tuple into name, marks, and attendance.
4. Display the student's name, marks, and attendance.
5. If marks are below 40 or attendance is below 50, set risk to "High Risk".
6. Otherwise, if marks are below 60 or attendance is below 75, set risk to "Medium Risk".
7. Otherwise, set risk to "Low Risk".
8. Display the academic risk of the student.
9. Repeat steps 3 to 8 for all students.
10. Display "Classification Completed!".
11. Stop the program.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

CREATE list of students (name, marks, attendance)

FOR each student in students:
    UNPACK student into name, marks, attendance
    DISPLAY name, marks, attendance

    IF marks < 40 OR attendance < 50:
        risk = "High Risk"
    ELSE IF marks < 60 OR attendance < 75:
        risk = "Medium Risk"
    ELSE:
        risk = "Low Risk"

    DISPLAY risk

DISPLAY "Classification Completed!"

STOP
```

7. Python Source Code / Program

```
# Student Academic Risk Classification

students = [
    ("Ravi", 85, 90),
    ("Priya", 65, 80),
    ("Raju", 45, 60),
    ("Sita", 30, 40),
    ("Kiran", 55, 45)
]

print("STUDENT ACADEMIC RISK CLASSIFICATION")
print("-" * 45)

for name, marks, attendance in students:
    print("\nStudent Name:", name)
    print("Marks:", marks)
    print("Attendance:", attendance, "%")

    if marks < 40 or attendance < 50:
        risk = "High Risk"
    elif marks < 60 or attendance < 75:
        risk = "Medium Risk"
    else:
        risk = "Low Risk"

    print("Academic Risk:", risk)

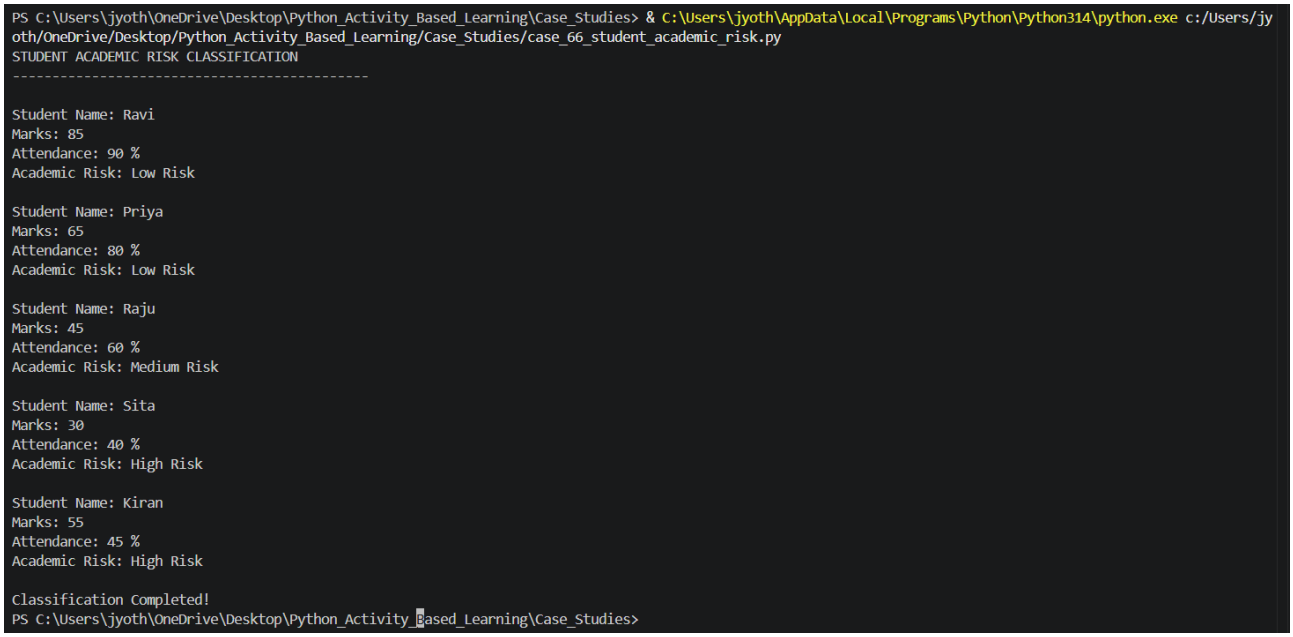
print("\nClassification Completed!")
```

8. Test Cases

Test case	Student (name, marks, attendance)	Input condition	Expected result
TC01	("Ravi", 85, 90)	Marks >= 60 and attendance >= 75	Low Risk
TC02	("Priya", 65, 80)	Marks >= 60 and attendance >= 75	Low Risk
TC03	("Raju", 45, 60)	Marks 40-59 and attendance 50-74	Medium Risk
TC04	("Sita", 30, 40)	Marks < 40 and attendance < 50	High Risk
TC05	("Kiran", 55, 45)	Attendance < 50 although marks >= 40	High Risk

Overall expected result: Low Risk = 2, Medium Risk = 1, High Risk = 2.

9. Output Screenshot



10. Conclusion

This program classifies students into High, Medium, and Low academic risk using marks and attendance. It unpacks tuple records, applies ordered conditions with the or operator, and displays a clear result for each student. The output matches the expected results of all five test cases, showing how simple rule-based checks can help identify students who need early academic support.

CASE 7: Integrated Student Record Consistency Checker

1. Problem Statement

Student records are used for results, attendance reports, and academic decisions, so they must be accurate and consistent. Missing names, marks outside the possible range, or impossible attendance percentages can lead to wrong reports. This project develops a Python program that stores each student record as a tuple (name, marks, attendance), checks every record against consistency rules, labels it as valid or invalid with a clear reason, and displays a summary of the results.

2. Objectives

1. To develop a Python program that checks the consistency of student records stored as tuples.
2. To unpack each tuple into name, marks, and attendance and display the details.
3. To detect records in which the student name is missing.
4. To detect marks that are outside the range 0 to 100.
5. To detect attendance percentages that are outside the range 0 to 100.
6. To display the status of every record and count the valid and invalid records.

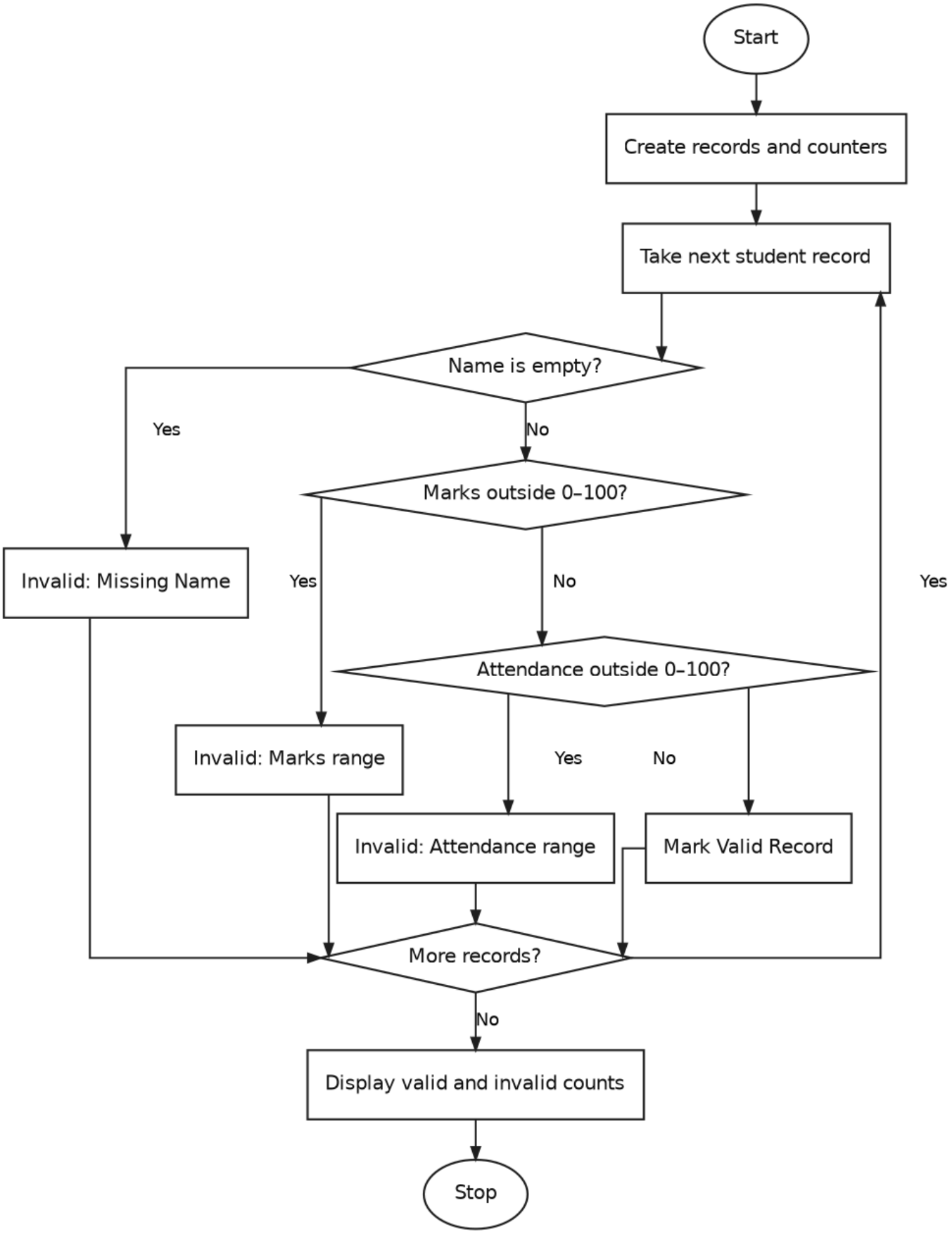
3. Assumptions

1. Each student record is a tuple in the form (name, marks, attendance).
2. A name is valid only if it is not an empty string; a missing name is displayed as "Missing".
3. Marks are valid if they lie between 0 and 100 (both inclusive).
4. Attendance is valid if it lies between 0 and 100 percent (both inclusive).
5. The checks are applied in the order name, marks, attendance, so only the first failed rule is reported.
6. A record is valid only when all three checks are satisfied.
7. Each record is counted exactly once, as either valid or invalid.
8. The student data is predefined in the program.

4. Algorithm

1. Start the program.
2. Create the list of student tuples and set valid_count and invalid_count to zero.
3. Take the next student and unpack the tuple into name, marks, and attendance.
4. Display the name (or "Missing" if empty), marks, and attendance.
5. If the name is empty, display "Invalid - Name is missing" and increase invalid_count.
6. Otherwise, if marks are below 0 or above 100, display "Invalid - Marks must be 0 to 100" and increase invalid_count.
7. Otherwise, if attendance is below 0 or above 100, display "Invalid - Attendance must be 0 to 100" and increase invalid_count.
8. Otherwise, display "Valid Record" and increase valid_count.
9. Repeat steps 3 to 8 for all students.
10. Display the valid, invalid, and total record counts.
11. Stop the program.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

CREATE list of students (name, marks, attendance)
SET valid_count = 0, invalid_count = 0

FOR each student in students:
    UNPACK student into name, marks, attendance
    DISPLAY name (or "Missing" if empty), marks, attendance

    IF name is empty:
        DISPLAY "Invalid - Name is missing"
        invalid_count = invalid_count + 1
    ELSE IF marks < 0 OR marks > 100:
        DISPLAY "Invalid - Marks must be 0 to 100"
        invalid_count = invalid_count + 1
    ELSE IF attendance < 0 OR attendance > 100:
        DISPLAY "Invalid - Attendance must be 0 to 100"
        invalid_count = invalid_count + 1
    ELSE:
        DISPLAY "Valid Record"
        valid_count = valid_count + 1

DISPLAY valid_count, invalid_count
DISPLAY total number of records

STOP
```

7. Python Source Code / Program

```
# Integrated Student Record Consistency Checker

students = [
    ("Ravi", 85, 90),
    ("Priya", 75, 80),
    ("Raju", 110, 70),
    ("Sita", 65, 105),
    ("", 80, 85)
]

print("STUDENT RECORD CONSISTENCY CHECKER")
print("-" * 40)

valid_count = 0
invalid_count = 0

for name, marks, attendance in students:
    print("\nStudent Name:", name if name else "Missing")
    print("Marks:", marks)
    print("Attendance:", attendance, "%")

    if name == "":
        print("Status: Invalid - Name is missing")
        invalid_count += 1
    elif marks < 0 or marks > 100:
        print("Status: Invalid - Marks must be 0 to 100")
        invalid_count += 1
    elif attendance < 0 or attendance > 100:
        print("Status: Invalid - Attendance must be 0 to 100")
        invalid_count += 1
    else:
        print("Status: Valid Record")
        valid_count += 1

print("\n" + "-" * 40)
print("Valid Records:", valid_count)
print("Invalid Records:", invalid_count)
print("Total Records:", len(students))
```

8. Test Cases

Test case	Student (name, marks, attendance)	Input condition	Expected result
TC01	("Ravi", 85, 90)	Name present; marks and attendance in range	Valid Record
TC02	("Priya", 75, 80)	Name present; marks and attendance in range	Valid Record
TC03	("Raju", 110, 70)	Marks above 100	Invalid - Marks must be 0 to 100
TC04	("Sita", 65, 105)	Attendance above 100	Invalid - Attendance must be 0 to 100
TC05	("", 80, 85)	Name is empty	Invalid - Name is missing

Overall expected result: Valid = 2, Invalid = 3, Total = 5.

9. Output Screenshot

```
PS C:\Users\jyoth\OneDrive\Desktop\Python_Activity_Based_Learning\Case_Studies> & C:/Users/jyoth/AppData/Local/Programs/Python/Python314/python.exe c:/Users/jyoth/OneDrive/Desktop/Python_Activity_Based_Learning/Case_Studies/case_75_student_record_checker.py
STUDENT RECORD CONSISTENCY CHECKER
----------------------------------------

Student Name: Ravi
Marks: 85
Attendance: 90 %
Status: Valid Record

Student Name: Priya
Marks: 75
Attendance: 80 %
Status: Valid Record

Student Name: Raju
Marks: 110
Attendance: 70 %
Status: Invalid - Marks must be 0 to 100

Student Name: Sita
Marks: 65
Attendance: 105 %
Status: Invalid - Attendance must be 0 to 100

Student Name: Missing
Marks: 80
Attendance: 85 %
Status: Invalid - Name is missing

----------------------------------------
Valid Records: 2
Invalid Records: 3
Total Records: 5
PS C:\Users\jyoth\OneDrive\Desktop\Python_Activity_Based_Learning\Case_Studies>
```

10. Conclusion

This program checks student records for a missing name, marks outside 0 to 100, and attendance outside 0 to 100. It unpacks tuple records, applies ordered conditions, reports a clear reason for every invalid record, and counts the valid and invalid records. The output matches the expected results of all five test cases, showing how simple validation rules improve the reliability of student data.

CASE 8: Smart Classroom Timetable Allocation

1. Problem Statement

Colleges must schedule many classes in a limited number of rooms. If two classes are given the same room on the same day at the same time, a clash occurs and one class cannot be held. Checking the timetable manually is slow and error-prone. This project develops a Python program that stores each timetable entry as a tuple (day, time, subject, room), checks every entry against the earlier entries, reports whether the room is available or already allocated, and counts the room conflicts.

2. Objectives

1. To develop a Python program that checks a classroom timetable for room conflicts.
2. To store each timetable entry as a tuple of day, time, subject, and room.
3. To unpack each entry and display its day, time, subject, and room.
4. To compare each entry with all earlier entries using nested loops.
5. To detect a conflict when the day, time, and room are all the same as an earlier entry.
6. To display the status of each entry and the total number of entries and room conflicts.

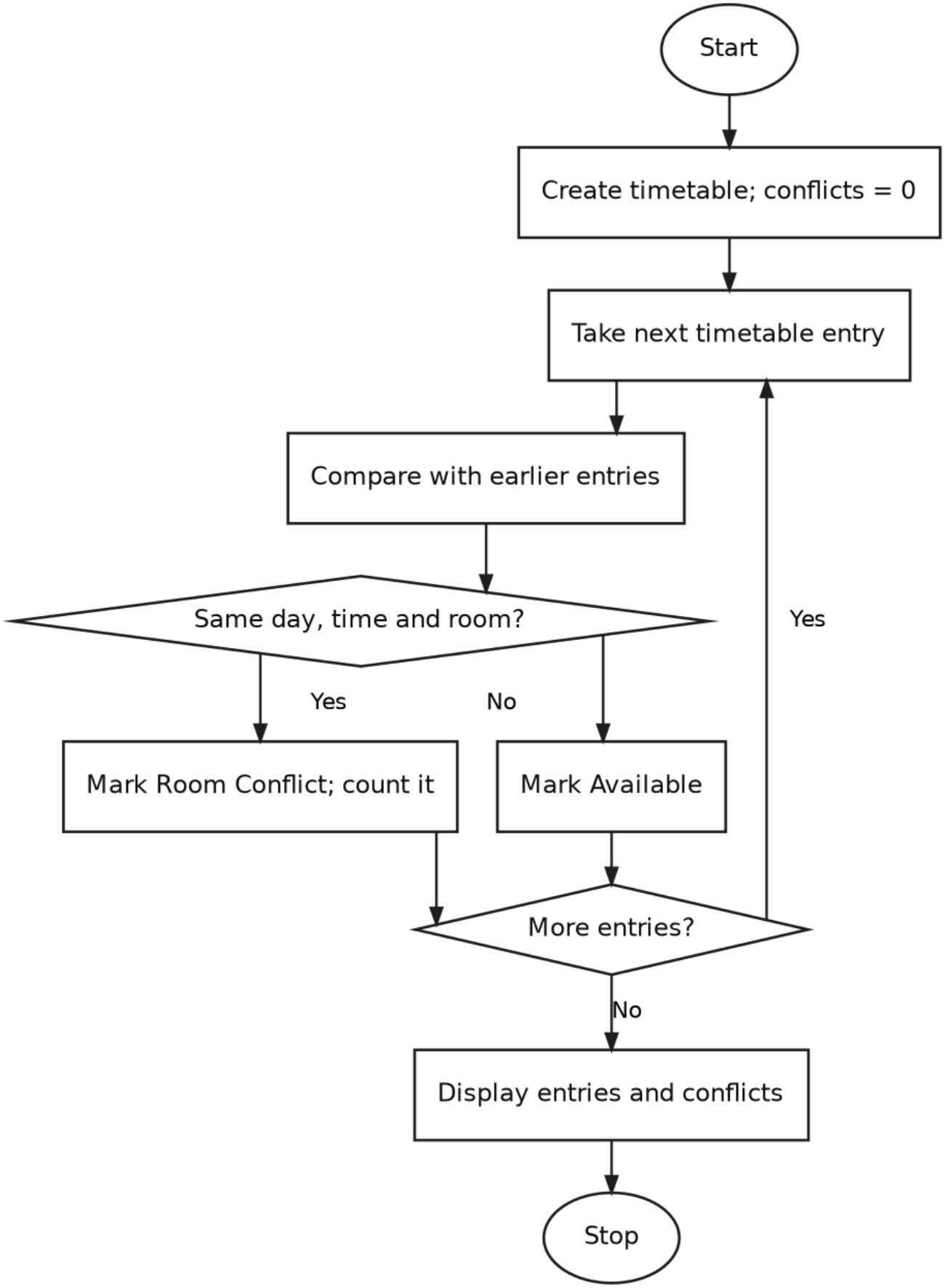
3. Assumptions

1. Each timetable entry is a tuple in the form (day, time, subject, room).
2. A room conflict occurs when two entries have the same day, the same time, and the same room.
3. The subject name is not used to detect a conflict.
4. Day, time, and room values must match exactly to be treated as the same.
5. Each entry is compared only with the entries that come before it, so the earlier entry keeps the room.
6. The later entry in a clash is marked as a conflict and counted once.
7. All entries are displayed, including those that have a conflict.
8. The timetable data is predefined in the program.

4. Algorithm

1. Start the program.
2. Create the timetable list of tuples and set conflicts to zero.
3. Take the next entry (index i) and unpack it into day, time, subject, and room.
4. Display the day, time, subject, and room.
5. Set conflict_found to False.
6. Compare the entry with every earlier entry (index j less than i).
7. If an earlier entry has the same day, time, and room, set conflict_found to True.
8. If conflict_found is True, display "Conflict - Room already allocated" and increase conflicts.
9. Otherwise, display "Available".
10. Repeat steps 3 to 9 for all entries.
11. Display the total entries, the number of room conflicts, and "Timetable Check Completed!".
12. Stop the program.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

CREATE timetable list of (day, time, subject, room)
SET conflicts = 0

FOR i = 0 TO number of entries - 1:
    UNPACK timetable[i] into day, time, subject, room
    DISPLAY day, time, subject, room

    SET conflict_found = False

    FOR j = 0 TO i - 1:
        UNPACK timetable[j] into old_day, old_time, old_subject, old_room
        IF day = old_day AND time = old_time AND room = old_room:
            SET conflict_found = True

    IF conflict_found:
        DISPLAY "Conflict - Room already allocated"
        conflicts = conflicts + 1
    ELSE:
        DISPLAY "Available"

DISPLAY total number of entries
DISPLAY conflicts
DISPLAY "Timetable Check Completed!"

STOP
```

7. Python Source Code / Program

```
# Smart Classroom Timetable Allocation

timetable = [
    ("Monday", "09:00 AM", "Python", "Room 101"),
    ("Monday", "10:00 AM", "Mathematics", "Room 101"),
    ("Monday", "09:00 AM", "English", "Room 102"),
    ("Tuesday", "09:00 AM", "Python Lab", "Room 101"),
    ("Tuesday", "09:00 AM", "AI", "Room 101")
]

print("SMART CLASSROOM TIMETABLE")
print("-" * 45)

conflicts = 0

for i in range(len(timetable)):
    day, time, subject, room = timetable[i]

    print("\nDay:", day)
    print("Time:", time)
    print("Subject:", subject)
    print("Room:", room)

    conflict_found = False

    for j in range(i):
        old_day, old_time, old_subject, old_room = timetable[j]

        if day == old_day and time == old_time and room == old_room:
            conflict_found = True

    if conflict_found:
        print("Status: Conflict - Room already allocated")
        conflicts += 1
    else:
        print("Status: Available")

print("\n" + "-" * 45)
print("Total Timetable Entries:", len(timetable))
print("Room Conflicts:", conflicts)
print("Timetable Check Completed!")
```

8. Test Cases

Test case	Entry (day, time, subject, room)	Input condition	Expected result
TC01	(Monday, 09:00 AM, Python, Room 101)	First entry in the timetable	Available
TC02	(Monday, 10:00 AM, Mathematics, Room 101)	Same room and day, different time	Available
TC03	(Monday, 09:00 AM, English, Room 102)	Same day and time, different room	Available
TC04	(Tuesday, 09:00 AM, Python Lab, Room 101)	Same time and room, different day	Available
TC05	(Tuesday, 09:00 AM, AI, Room 101)	Same day, time, and room as TC04	Conflict - Room already allocated

Overall expected result: Total entries = 5, Room conflicts = 1.

9. Output Screenshot

```
PS C:\Users\jyoth\OneDrive\Desktop\Python_Activity_Based_Learning\Case_Studies> & C:/Users/jyoth/AppData/Local/Programs/Python/Python314/python.exe c:/Users/jyoth/OneDrive/Desktop/Python_Activity_Based_Learning/Case_Studies/case_85_smart_classroom_timetable.py
SMART CLASSROOM TIMETABLE
--------------------------------------------

Day: Monday
Time: 09:00 AM
Subject: Python
Room: Room 101
Status: Available

Day: Monday
Time: 10:00 AM
Subject: Mathematics
Room: Room 101
Status: Available

Day: Monday
Time: 09:00 AM
Subject: English
Room: Room 102
Status: Available

Day: Tuesday
Time: 09:00 AM
Subject: Python Lab
Room: Room 101
Status: Available

Day: Tuesday
Time: 09:00 AM
Subject: AI
Room: Room 101
Status: Conflict - Room already allocated

--------------------------------------------
Total Timetable Entries: 5
Room Conflicts: 1
Timetable Check Completed!
PS C:\Users\jyoth\OneDrive\Desktop\Python_Activity_Based_Learning\Case_Studies>
```

10. Conclusion

This program checks a classroom timetable for room conflicts. It unpacks each tuple entry, compares it with all earlier entries using nested loops, and reports whether the room is available or already allocated. The output matches the expected results of all five test cases, showing how nested loops and tuple unpacking can detect scheduling clashes automatically.

CASE 9: Advanced String-Based Registration Record Analyzer

1. Problem Statement

Colleges maintain registration records in which every student has a registration ID and a name. Errors such as repeated IDs, IDs in the wrong format, and missing names make the records unreliable. This project develops a Python program that stores each record as a tuple (registration ID, name), uses string methods to check the ID format and the name, detects duplicate registration IDs, classifies every record as valid, invalid, or duplicate, and displays a summary of the analysis.

2. Objectives

1. To develop a Python program that analyzes registration records stored as tuples.
2. To unpack each tuple into registration ID and student name and display the details.
3. To detect duplicate registration IDs using a list of IDs already seen.
4. To check the ID format using the string method startswith("REG").
5. To detect missing student names using the string method strip().
6. To display the status of every record and count valid, invalid, and duplicate records.

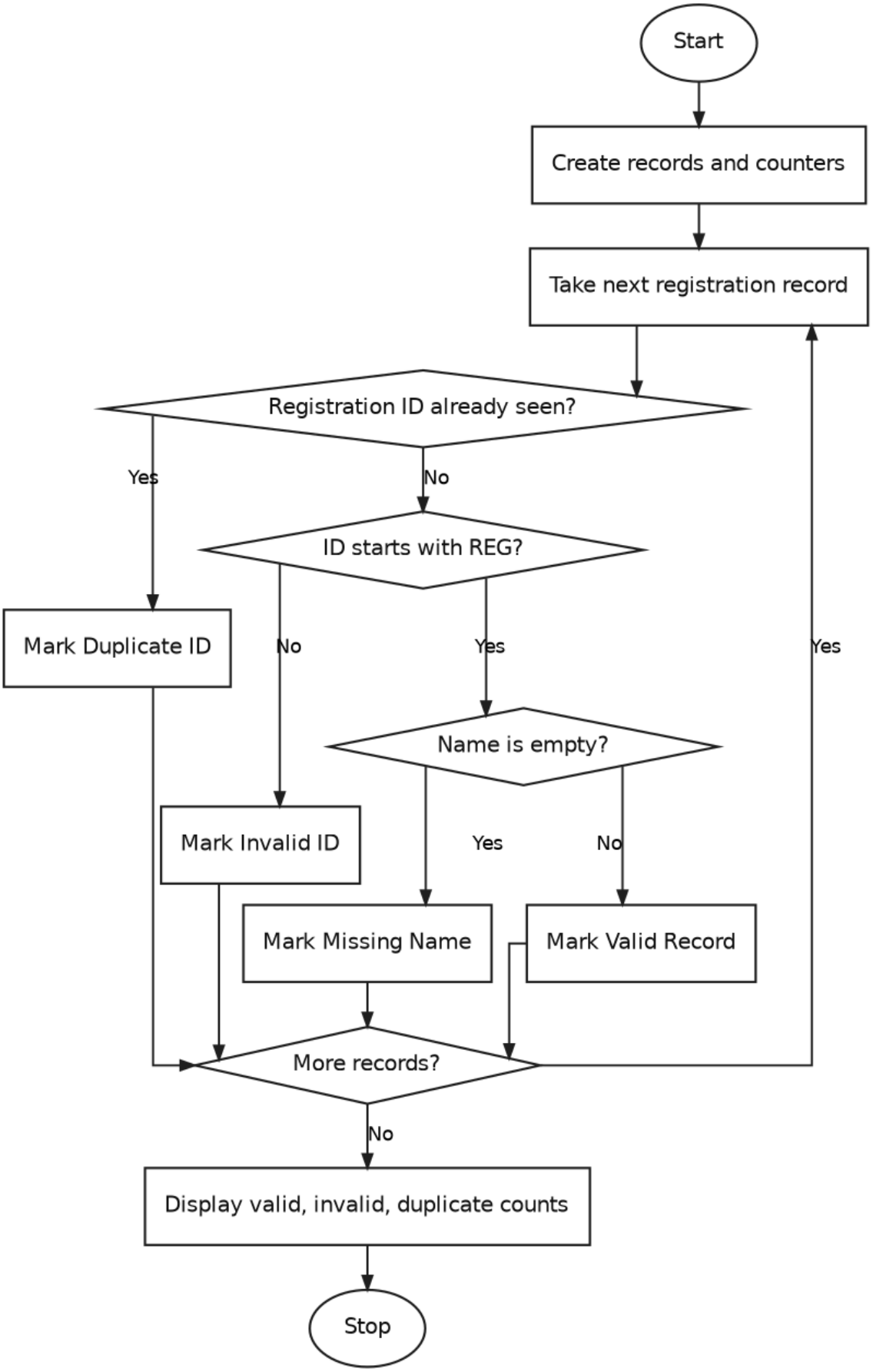
3. Assumptions

1. Each record is a tuple in the form (registration_id, student_name).
2. A valid registration ID must start with "REG" (case-sensitive).
3. A student name is missing if it is empty or contains only spaces; it is displayed as "Missing".
4. The checks are applied in the order duplicate ID, ID format, student name.
5. The first occurrence of a registration ID is processed; later records with the same ID are duplicates.
6. IDs of valid and invalid records are stored in `seen_ids`; duplicate records are not stored again.
7. Each record is counted exactly once as valid, invalid, or duplicate.
8. The record data is predefined in the program.

4. Algorithm

1. Start the program.
2. Create the list of record tuples and an empty list `seen_ids`.
3. Set valid_count, invalid_count, and duplicate_count to zero.
4. Take the next record and unpack it into registration ID and name; display them (name shown as "Missing" if empty).
5. If the ID is already in seen_ids, display "Duplicate Registration ID" and increase duplicate_count.
6. Otherwise, if the ID does not start with "REG", display "Invalid Registration ID", increase invalid_count, and store the ID.
7. Otherwise, if the name is empty after strip(), display "Missing Student Name", increase invalid_count, and store the ID.
8. Otherwise, display "Valid Record", increase valid_count, and store the ID.
9. Repeat steps 4 to 8 for all records.
10. Display the valid, invalid, duplicate, and total record counts.
11. Stop the program.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

CREATE list of records (reg_id, name)
CREATE empty list seen_ids
SET valid_count = 0, invalid_count = 0, duplicate_count = 0

FOR each record in records:
    UNPACK record into reg_id, name
    DISPLAY reg_id
    DISPLAY name (or "Missing" if name is empty)

    IF reg_id is in seen_ids:
        DISPLAY "Duplicate Registration ID"
        duplicate_count = duplicate_count + 1
    ELSE IF reg_id does not start with "REG":
        DISPLAY "Invalid Registration ID"
        invalid_count = invalid_count + 1
        ADD reg_id to seen_ids
    ELSE IF name is empty after removing spaces:
        DISPLAY "Missing Student Name"
        invalid_count = invalid_count + 1
        ADD reg_id to seen_ids
    ELSE:
        DISPLAY "Valid Record"
        valid_count = valid_count + 1
        ADD reg_id to seen_ids

DISPLAY valid_count, invalid_count, duplicate_count
DISPLAY total number of records

STOP
```

7. Python Source Code / Program

```
# Advanced String-Based Registration Record Analyzer
students = [
    ("REG101", "Ravi"),
    ("REG102", "Priya"),
    ("REG103", ""),
    ("REG102", "Raju"),
    ("ABC104", "Sita")
]
seen_ids = []
valid_count = 0
invalid_count = 0
duplicate_count = 0
print("REGISTRATION RECORD ANALYZER")
print("-" * 40)
for reg_id, name in students:
    print("\nRegistration ID:", reg_id)
    print("Student Name:", name if name else "Missing")
    if reg_id in seen_ids:
        print("Status: Duplicate Registration ID")
        duplicate_count += 1

    elif not reg_id.startswith("REG"):
        print("Status: Invalid Registration ID")
        invalid_count += 1
        seen_ids.append(reg_id)

    elif name.strip() == "":
        print("Status: Missing Student Name")
        invalid_count += 1
        seen_ids.append(reg_id)

    else:
        print("Status: Valid Record")
        valid_count += 1
        seen_ids.append(reg_id)

print("\n" + "-" * 40)
print("Valid Records:", valid_count)
print("Invalid Records:", invalid_count)
print("Duplicate Records:", duplicate_count)
print("Total Records:", len(students))
```

8. Test Cases

Test case	Record (reg_id, name)	Input condition	Expected result
TC01	("REG101", "Ravi")	New ID starting with REG; name present	Valid Record
TC02	("REG102", "Priya")	New ID starting with REG; name present	Valid Record
TC03	("REG103", "")	Name is empty	Missing Student Name
TC04	("REG102", "Raju")	ID REG102 already processed	Duplicate Registration ID
TC05	("ABC104", "Sita")	ID does not start with REG	Invalid Registration ID

Overall expected result: Valid = 2, Invalid = 2, Duplicate = 1, Total = 5.

9. Output Screenshot

```
PS C:\Users\jyoth\OneDrive\Desktop\Python_Activity_Based_Learning\Case_Studies> & C:/Users/jyoth/AppData/Local/Programs/Python/Python314/python.exe c:/Users/jyoth/OneDrive/Desktop/Python_Activity_Based_Learning/Case_Studies/case_91_registration_record_anayzer.py
REGISTRATION RECORD ANALYZER
----------------------------------------

Registration ID: REG101
Student Name: Ravi
Status: Valid Record

Registration ID: REG102
Student Name: Priya
Status: Valid Record

Registration ID: REG103
Student Name: Missing
Status: Missing Student Name

Registration ID: REG102
Student Name: Raju
Status: Duplicate Registration ID

Registration ID: ABC104
Student Name: Sita
Status: Invalid Registration ID

----------------------------------------
Valid Records: 2
Invalid Records: 2
Duplicate Records: 1
Total Records: 5
PS C:\Users\jyoth\OneDrive\Desktop\Python_Activity_Based_Learning\Case_Studies>
```

10. Conclusion

This program analyzes registration records using tuple unpacking, a list of seen IDs, and the string methods startswith() and strip(). It detects duplicate IDs, invalid ID formats, and missing names, and reports the number of valid, invalid, and duplicate records. The output matches the expected results of all five test cases, showing how string handling and conditions can keep registration data consistent.

CASE 10: Comprehensive Smart Campus Management Simulation

1. Problem Statement

A college campus needs to monitor several services together: student attendance, library book availability, and classroom status. Handling each report separately is slow and makes it hard to see the overall position of the campus. This project develops a Python program that stores student, library, and classroom records as lists of tuples, processes each list with loops and conditions, prints a separate report for each service, and finishes with a combined campus summary.

2. Objectives

1. To develop a Python program that simulates the management of a smart campus.
2. To store student attendance, library books, and classroom status as lists of tuples.
3. To check each student's attendance and identify students who are eligible (75% or above).
4. To display each library book as available or issued and count the available books.
5. To display each classroom's status and count the available classrooms.
6. To display a final campus summary of all counts.

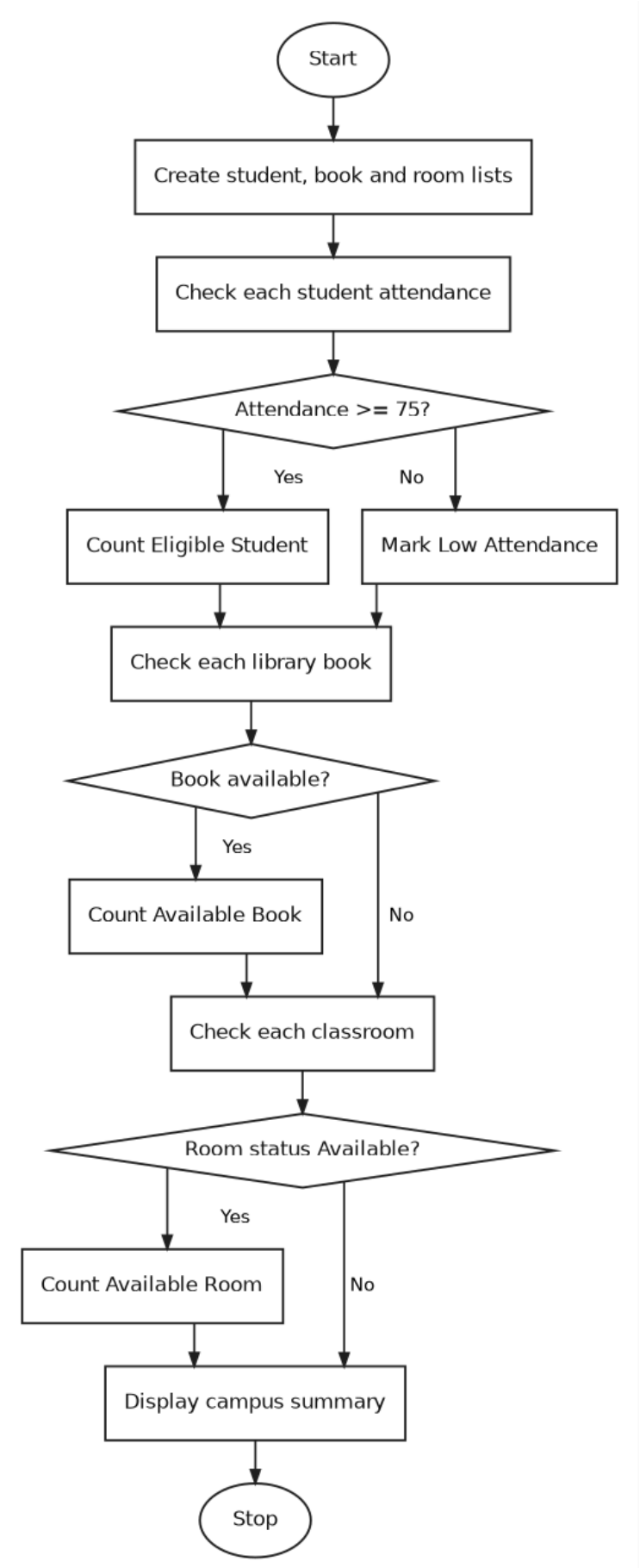
3. Assumptions

1. Each student record is a tuple in the form (name, attendance_percentage).
2. A student is eligible by attendance if attendance is 75% or more.
3. Each library record is a tuple (book_title, available), where True means the book is available and False means it is issued.
4. Each classroom record is a tuple (room, status), where the status is "Available" or "Occupied".
5. Only the status "Available" (exact match) is counted as an available classroom.
6. The three reports are produced one after another, followed by the final summary.
7. The campus data is predefined in the program and is entered correctly.

4. Algorithm

1. Start the program.
2. Create the lists of tuples for students, library books, and classrooms.
3. Display the system title. Set present_count to zero.
4. For each student, display the name and attendance.
5. If attendance is 75 or more, display "Eligible" and increase present_count; otherwise display "Low Attendance".
6. Set available_books to zero.
7. For each book, display "Available" and increase available_books if it is available; otherwise display "Issued".
8. Set available_rooms to zero.
9. For each classroom, display the room and its status; if the status is "Available", increase available_rooms.
10. Display the total students, students eligible by attendance, available books, and available classrooms.
11. Display "Campus Report Completed!".
12. Stop the program.

5. Flowchart



MEHAVARSHINI

6. Pseudocode

```
START

CREATE students list of (name, attendance)
CREATE library_books list of (book, available)
CREATE classrooms list of (room, status)

DISPLAY "SMART CAMPUS MANAGEMENT SYSTEM"

SET present_count = 0
FOR each (name, attendance) in students:
    DISPLAY name, attendance
    IF attendance >= 75:
        DISPLAY "Status: Eligible"
        present_count = present_count + 1
    ELSE:
        DISPLAY "Status: Low Attendance"

SET available_books = 0
FOR each (book, available) in library_books:
    IF available:
        DISPLAY book, "Available"
        available_books = available_books + 1
    ELSE:
        DISPLAY book, "Issued"

SET available_rooms = 0
FOR each (room, status) in classrooms:
    DISPLAY room, status
    IF status = "Available":
        available_rooms = available_rooms + 1

DISPLAY total students
DISPLAY present_count, available_books, available_rooms
DISPLAY "Campus Report Completed!"

STOP
```

7. Python Source Code / Program

```
# Comprehensive Smart Campus Management Simulation

students = [
    ("Ravi", 90),
    ("Priya", 75),
    ("Raju", 45),
    ("Sita", 95)
]

library_books = [
    ("Python Basics", True),
    ("Web Development", False),
    ("AI Fundamentals", True)
]

classrooms = [
    ("Room 101", "Available"),
    ("Room 102", "Occupied"),
    ("Room 103", "Available")
]

print("SMART CAMPUS MANAGEMENT SYSTEM")
print("=" * 40)

# Student Attendance Report
print("\nSTUDENT ATTENDANCE REPORT")
present_count = 0

for name, attendance in students:
    print(name, "-", attendance, "% attendance")

    if attendance >= 75:
        print("Status: Eligible")
        present_count += 1
    else:
        print("Status: Low Attendance")

# Library Report
print("\nLIBRARY BOOK REPORT")
available_books = 0

for book, available in library_books:
    if available:
        print(book, "- Available")
        available_books += 1
    else:
        print(book, "- Issued")

# Classroom Report
print("\nCLASSROOM STATUS REPORT")
available_rooms = 0

for room, status in classrooms:
    print(room, "-", status)

    if status == "Available":
        available_rooms += 1

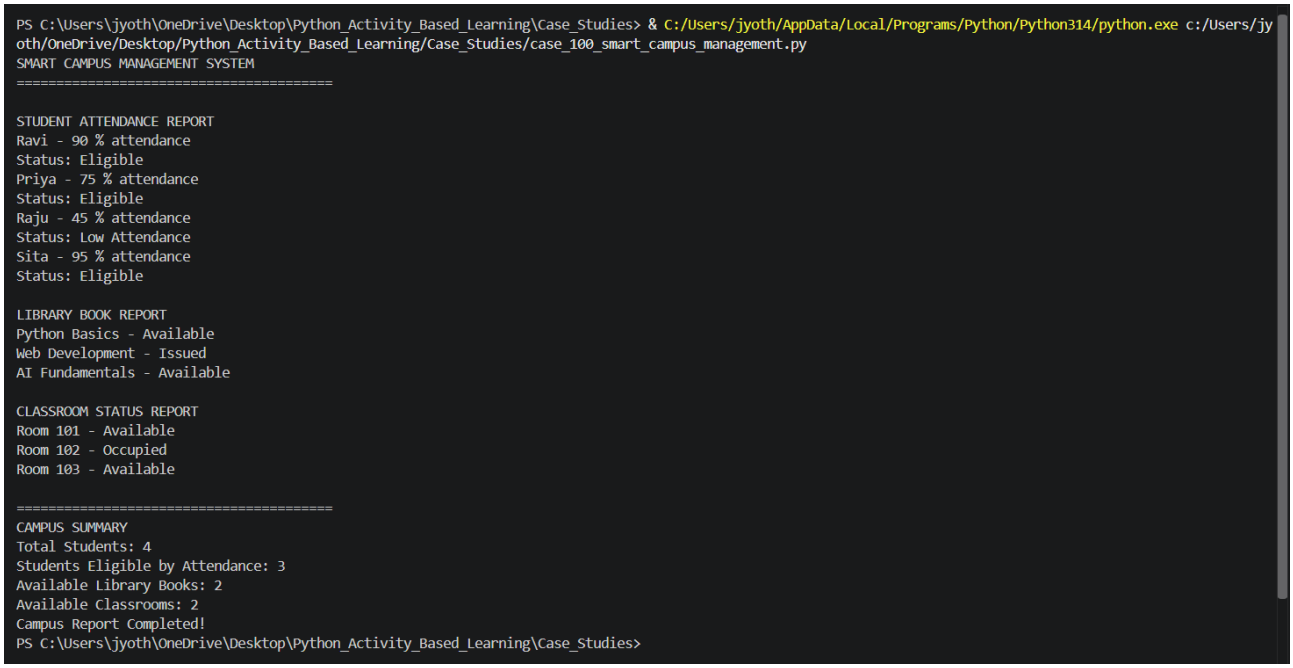
# Final Summary
print("\n" + "=" * 40)
print("CAMPUS SUMMARY")
print("Total Students:", len(students))
print("Students Eligible by Attendance:", present_count)
print("Available Library Books:", available_books)
print("Available Classrooms:", available_rooms)
print("Campus Report Completed!")
```

8. Test Cases

Test case	Record	Input condition	Expected result
TC01	Student ("Ravi", 90)	Attendance above 75	Status: Eligible
TC02	Student ("Priya", 75)	Attendance exactly 75 (boundary)	Status: Eligible
TC03	Student ("Raju", 45)	Attendance below 75	Status: Low Attendance
TC04	Book ("Web Development", False)	Book is not available	Web Development - Issued
TC05	Classroom ("Room 102", "Occupied")	Status is not "Available"	Shown as Occupied; not counted

Overall expected result: Total students = 4, Students eligible by attendance = 3, Available library books = 2, Available classrooms = 2.

9. Output Screenshot



10. Conclusion

This program combines attendance checking, library book tracking, and classroom status monitoring in one campus report. It uses lists of tuples, loops, conditions, and counters, and ends with a clear summary of eligible students, available books, and available classrooms. The output matches the expected results of all five test cases, showing how basic Python constructs can be integrated into a simple management system.